LEÇON PRATIQUE · JAVA 21 / 25 · MISE À JOUR NOVEMBRE 2026

Entrées, limites et erreurs

Cadrer toute saisie, imposer des plafonds (5 jours, 5 objets, 1 parité), et gérer les erreurs sans les avaler — avec les exceptions du JDK, pas contre elles.

Public	Développeuse / développeur Java qui écrit du code qui reçoit quelque chose de l'extérieur : un formulaire, un fichier, une ligne de commande, une requête HTTP, une sauvegarde de jeu. Niveau intermédiaire.
Prérequis	Un JDK 21 ou 25 (`javac -version` répond), un terminal, savoir compiler un fichier `.java`. Aucune bibliothèque externe : tout tient dans le JDK.
Durée	≈ 2 h de lecture + 45 min de reproduction des exemples + 45 min d'exercices (chapitre 13).
Objectif final	Répondre sans hésiter à : « où je valide cette valeur ? », « quelle exception je lève ? », « comment je refuse 6 objets au lieu de 5 ? », « comment signaler trois erreurs d'un coup ? ».

valider à la frontière · Math.addExact · BigDecimal · LocalDate · Clock · Period · quota · rate limiting · exceptions typées · cause · in : Scanner · BufferedReader · args · out : format · CSV · encodage · config : properties · YAML · TOML · JLS §4.2.2 · §14.10 · §14.20.3

Les trois questions de départ, et leurs réponses courtes

« Comment imposer une limite de 5 jours ou de 5 objets ? » **Par un type qui ne peut pas être faux.** Une limite n'est pas un `if` perdu dans un contrôleur : c'est un invariant du constructeur (section 1.3), doublé d'une vérification à la frontière (chapitre 4) et, si l'entrée est hostile, d'un plafond technique (chapitre 5).

« Comment bien gérer les erreurs ? » **En échouant tôt, avec le bon type d'exception, et sans jamais perdre la cause.** Valider tout ce qui peut l'être avant de construire, accumuler les erreurs de saisie pour les rendre toutes d'un coup, envelopper les erreurs techniques sans les écraser (chapitre 7).

« Et cadrer l'output ? » **Un seul format, fixé et testé** : longueurs stables, délimiteurs échappés, encodage explicite, message utilisateur séparé du message technique (chapitre 11).

Sources principales : JLS SE 25 — §4.2.2, §5.1.3, §11, §14.10, §14.20.3, §15.17.2 · Javadoc du JDK 25 : `Math`, `Integer`, `BigDecimal`, `LocalDate`, `DateTimeFormatter`, `Clock`, `Objects`, `Scanner`, `BufferedReader`, `CharsetDecoder`, `Formatter`, `Charset`, `Files`, `Console`, `Properties`, `SecureRandom`, `Normalizer` · RFC 4180 (CSV), RFC 9457 (erreurs d'API), CVE-2022-1471 (SnakeYAML). Toutes les sorties ont été exécutées sur Debian 13, Temurin 25.0.5, en novembre 2026.

Sommaire

1 Le modèle mental : entrée → validation → objet valide → traitement 5
- 1.1 Le bug n'est presque jamais là où on le cherche 5
- 1.2 Les huit frontières par lesquelles le monde entre dans votre code 5
- 1.3 La règle centrale : un objet construit est un objet valide 6
- 1.4 Deux limites, à ne jamais confondre : métier et technique 7
- 1.5 La grille de décision, en une page 8

2 Les limites numériques 9
- 2.1 Les bornes des types entiers, et le débordement silencieux 9
- 2.2 `Math.*Exact` : rendre l'échec visible 9
- 2.3 Le piège de `Math.abs(Integer.MIN_VALUE)` 10
- 2.4 Division, modulo et signes 11
- 2.5 Flottants : arrondi, Infinity, NaN 11
- 2.6 L'argent : centimes entiers ou `BigDecimal` 12
- 2.7 Convertir une saisie : `parseInt` et ses pièges 13

3 Les limites de dates et de durées 15
- 3.1 `Clock` : le temps doit être injectable 15
- 3.2 Compter des jours : inclusif, exclusif, calendaire 15
- 3.3 `plusDays(1)` n'est pas 24 heures 16
- 3.4 Les formats d'entrée : `uuuu`, `ResolverStyle`, et le 30 février 16
- 3.5 Plages, chevauchements et bornes du temps 18

4 Structurer la validation d'entrée 20
- 4.1 Valider et convertir : deux verbes différents 20
- 4.2 Accumuler les erreurs ou s'arrêter à la première 20
- 4.3 Le résultat de validation et ses quatre issues 21
- 4.4 Les six limites de saisie et leurs messages 22
- 4.5 Où placer chaque contrôle : le tableau de décision 23

5 Volumétrie : compter les objets, les octets, la profondeur 24
- 5.1 Collections : plafond métier et plafond technique 24
- 5.2 Listes immuables, `null` interdit, fuites évitées 25
- 5.3 Le vrai mur : `OutOfMemoryError` 26
- 5.4 Chaînes et JSON : longueur et profondeur 27
- 5.5 Fichiers et flux : la taille avant le contenu 28

6 Quotas, rythme et temps de traitement 30
- 6.1 Quota par utilisateur, par période 30
- 6.2 Fenêtre glissante et seau à jetons 30
- 6.3 Budgéter le temps de traitement 31
- 6.4 Files bornées : refuser plutôt que s'écrouler 32

7 Gérer les erreurs : lever, attraper, envelopper 33
- 7.1 Quand lever, quand retourner une valeur 33
- 7.2 Choisir parmi les exceptions du JDK 33
- 7.3 Une hiérarchie d'exceptions maison 34
- 7.4 Attraper correctement 35
- 7.5 Envelopper sans perdre la cause 37
- 7.6 Le coût d'une exception dans une boucle de jeu 38

8 Étude de cas : un jeu (saisie, stock, triche) 40

8.1 La boucle de saisie bornée 40
8.2 Stock, quantités et commandes par tour 41
8.3 Ne jamais croire le client 42
8.4 Hasard reproductible ou imprévisible 42
9 Lire une entrée : sources, lecteurs, plafonds 44
9.1 Le paysage : cinq sources, cinq lecteurs 44
9.2 `Scanner` : des tokens, un délimiteur, une position 45
9.3 Les quatre pièges de `Scanner` 46
9.4 `BufferedReader` et `Scanner` : lequel, et à quel prix 47
9.5 Chaîne, octets, encodage : décoder est une décision 47
9.6 Lire un fichier : taille, lignes, BOM, fins de ligne 48
9.7 `args` : la seule entrée qui vient sans mode d'emploi 49
9.8 Borner la lecture : un lecteur qui refuse le trop-gros 51
9.9 Console, `System.in` et fin de flux 52
9.10 Décider vite : huit lignes pour choisir son lecteur 54
10 Configuration : `.properties`, YAML, TOML 55
10.1 Ce que le format change : trois textes, trois résultats 55
10.2 YAML : les types que vous n'avez pas écrits 55
10.3 Les sept pièges de YAML, mesurés 56
10.4 TOML : typé, strict, et il vous dit où est la faute 57
10.5 `.properties` : le format du JDK, et ses quatre surprises 58
10.6 Quatre sources, une seule valeur : la précédence 59
10.7 Valider la configuration au démarrage 59
10.8 YAML hostile : la bombe et le tag 60
10.9 Décider vite : quel format, et avec quel garde-fou 61
11 Cadrer l'output 62
11.1 Un format unique, fixé et testé 62
11.2 `printf` : largeurs, précision, erreurs de format 62
11.3 Les délimiteurs : `Scanner` et le CSV 63
11.4 Encodage et Unicode : l'output qui devient illisible 64
11.5 Message utilisateur, message technique 66
12 Erreurs et pièges : les cas réels 67
13 Exercices corrigés et fiche de synthèse 71
14 Sources et pour aller plus loin 78

Le fil rouge : une demande de location, un jeu, un service

Une même logique, trois contextes, des exemples compilés et exécutés :

Contexte	Ce que montre l'exemple	Chapitre
Demande de location	Référence, date de début, **5 jours maximum**, **5 articles maximum**, 9 passagers, commentaire de 200 caractères. Lue depuis `args`, un fichier ou un flux, puis objet validé.	4 · 7 · 9
Boucle de jeu	Menu lu au clavier avec nombre d'essais borné, stock de potions, commandes par tour, score vérifié côté serveur.	6 · 8
Service / API	Configuration validée au démarrage (YAML/TOML), quotas par utilisateur, rate limiting, budget de temps, lecteur borné sur un flux réseau, format de réponse normalisé.	5 · 6 · 9 · 10 · 11

Rien de spectaculaire, et c'est voulu : les bugs de production ne viennent presque jamais d'un algorithme subtil. Ils viennent d'une valeur `-1` qui traverse trois couches, d'une date au 30 février acceptée par un parseur trop gentil, d'un montant `double` qui affiche 5.3999999999999995, ou d'un `catch (Exception e) {}` qui a masqué la panne. Ce document traite les quatre.

1 Le modèle mental : entrée → validation → objet valide → traitement

> Avant d'étudier un seul cas particulier, il faut une image mentale unique. Une fois qu'elle est en place, chaque limite du document (5 jours, 5 objets, 200 caractères, un quota par heure) devient une simple application du même schéma — et vous saurez toujours où écrire le contrôle.

1.1 Le bug n'est presque jamais là où on le cherche

Voici l'histoire, vécue chaque semaine dans le monde professionnel. Un service calcule un prix. Il fonctionne parfaitement en test, avec des données propres. En production, un client reçoit une facture de `-5.3999999999999995 €`. On cherche le bug dans la fonction de calcul — une quinzaine de lignes de multiplications. On n'y trouve rien de faux : le calcul est correct, c'est l'entrée qui était impossible.

PROGRAMME — LE MÊME CALCUL, DEUX ENTRÉES, DONT UNE IMPOSSIBLE

```
public class Prix {
    static double tva(double prixHT, int tauxPourCent) {
        return prixHT * (1 + tauxPourCent / 100.0);
    }

    public static void main(String[] args) {
        System.out.println("entrée normale   : " + tva(4.50, 20));
        System.out.println("entrée impossible: " + tva(-0.30, 20));
        System.out.println("arrondi          : " + (tva(4.50, 20) * 3));
    }
}
```

SORTIE RÉELLE

```
entrée normale   : 5.3999999999999995
entrée impossible: -0.36
arrondi          : 16.200000000000003
```

Trois enseignements dans ces trois lignes. **Un** : la fonction « correcte » produit déjà un résultat laid, parce qu'un `double` ne représente pas exactement 5,40 € — c'est un problème de type, traité en 2.5 et 2.6. **Deux** : elle accepte un prix négatif sans broncher, parce que rien ne l'interdit — c'est un problème de validation, traité en 1.3 puis au chapitre 4. **Trois** : le vrai coût n'est pas le calcul, c'est le temps passé à chercher la panne au mauvais endroit.

> **La question qui fait gagner des heures**
>
> Face à un résultat absurde, posez toujours la même question avant de lire le code de calcul : **cette valeur pouvait-elle entrer ?** Si oui, la correction ne consiste pas à ajouter un `if` près du calcul, mais à interdire la valeur à la frontière. C'est la différence entre un rustine et une correction.

1.2 Les huit frontières par lesquelles le monde entre dans votre code

Une « entrée » n'est pas forcément un champ de formulaire. Tout ce qui n'est pas une valeur littérale écrite dans votre code source est une entrée, donc une source de surprise. Les huit frontières à connaître :

Frontière	Ce qui peut arriver (cas réel)	Où valider
`String[] args`	Zéro argument, plus d'arguments que prévu, écriture d'un texte, d'un nombre, d'une date… et le programme continue	Au tout début de `main`, avant tout traitement (ch. 9.7)
Entrée console	Saisie non numérique, valeur hors menu, fin de flux (`Ctrl-D`) qui boucle à l'infini	Dans une boucle d'essais bornés (ch. 8.1)
Fichier	Absent, illisible, énorme, encodage différent, dernière ligne sans fin de ligne	Avant la lecture : taille, existence, encodage (ch. 5.5, 9.6)
Requête HTTP / API	Corps mal formé, champs absents, nombres en texte, taille arbitraire, payload profond	Dès l'entrée du contrôleur, avant le domaine (ch. 4, 5.4)
Base de données	Valeurs `null`, colonnes tronquées, dates invalides héritées d'un import	À la lecture : reconstruire un objet valide (1.3)
Configuration / env.	Variable absente, chaîne vide, entier en texte, valeur par défaut jamais vérifiée	Au démarrage de l'application, échouer vite
Désérialisation	JSON/CSV avec types inattendus, champs supplémentaires, valeurs `null`	Dans le convertisseur, jamais dans le domaine (ch. 11.3)
Votre propre API	Un collègue (ou vous, dans six mois) passe `null`, `-1`, une liste vide là où ce n'était pas prévu	Dans le constructeur public (1.3)

La ligne de partage

Les sept premières frontières se traitent **à l'entrée** : on convertit, on refuse, on explique. La huitième — votre propre API — se traite **dans le type** : c'est le sujet de la section suivante, et c'est la plus rentable, parce qu'elle protège aussi des six autres.

1.3 La règle centrale : un objet construit est un objet valide

Voici la règle autour de laquelle tout le document est construit :

Invariant de construction

Si un objet existe, alors tous ses invariants sont vrais. Aucune méthode n'a besoin de revérifier ; aucun appelant ne peut créer un objet faux, même en passant par un autre chemin. Le contrôle s'écrit **une fois**, dans le constructeur, et disparaît ensuite de tout le reste du code.

En Java moderne, l'outil idéal est le `record` avec son constructeur compact : toutes les vérifications y passent, puis les composants sont affectés automatiquement.

PROGRAMME — UNE LIMITE DE 5 OBJETS ET DE 5 JOURS, DANS LE TYPE

```
public record Commande(String reference, int articles, int jours) {

    public static final int ARTICLES_MAX = 5;
    public static final int JOURS_MAX = 5;

    public Commande {                       // constructeur compact : s'exécute à chaque création
        if (reference == null || reference.isBlank())
            throw new IllegalArgumentException("référence obligatoire");
        if (articles < 1 || articles > ARTICLES_MAX)
            throw new IllegalArgumentException("articles hors bornes : " + articles + " (1 à " +
ARTICLES_MAX + ")");
        if (jours < 1 || jours > JOURS_MAX)
```

```
            throw new IllegalArgumentException("jours hors bornes : " + jours + " (1 à " +
JOURS_MAX + ")");
    }
}

// et l'appelant : aucune vérification, jamais
Commande c = new Commande("LOC-2026-11", 5, 5);   // ok
Commande d = new Commande("LOC-2026-11", 6, 5);   // IllegalArgumentException
```

SORTIE RÉELLE

```
Commande[reference=LOC-2026-11, articles=5, jours=5]
refus : articles hors bornes : 6 (1 à 5)
refus : jours hors bornes : 0 (1 à 5)
refus : référence obligatoire
refus : articles hors bornes : -1 (1 à 5)
```

Comparez avec l'approche « contrôle dans le service » : trois méthodes de service, une par point d'entrée, et dans six mois une quatrième méthode écrite par quelqu'un qui n'a pas lu les trois premières. Le constructeur, lui, est impossible à contourner — sauf par réflexion, et par désérialisation, ce qui est précisément le sujet du chapitre 9.3.

> **Les quatre limites du `record` qu'il faut connaître**
>
> **1.** Un `record` ne peut pas être étendu : tant mieux, la validation reste unique. **2.** Ses composants sont accessibles en lecture : si un composant est une liste ou un tableau, il faut le copier à la construction et le renvoyer copié (section 5.2). **3.** La désérialisation d'un `record` passe bien par le constructeur canonique — la validation s'applique donc aussi aux objets relus. **4.** Une validation qui dépend de l'heure, du fuseau ou d'un référentiel ne peut pas être dans le constructeur : il faut alors la porter par un autre type, par exemple une `Plage` calculée à partir d'une horloge injectée (section 3.1).

1.4 Deux limites, à ne jamais confondre : métier et technique

« Limiter les entrées » recouvre deux préoccupations très différentes. Les mélanger produit soit des applications fragiles, soit des messages absurdes pour l'utilisateur.

Critère	Limite métier	Limite technique
Exemple	5 jours de location, 5 articles, 9 passagers, 200 caractères de commentaire	2 Go de tas, 1 Go par fichier, 100 niveaux de profondeur JSON, 5 000 requêtes par minute
Qui la fixe	Le produit, le client, la réglementation	La machine, la JVM, l'infrastructure
Peut-elle changer ?	Oui, du jour au lendemain, sur demande	Rarement, mais elle change quand le matériel change
Exception associée	`IllegalArgumentException` ou exception métier typée	`OutOfMemoryError`, `StackOverflowError`, réponse 413/429
Message à l'utilisateur	« La durée maximale est de 5 jours »	« Ce fichier dépasse la taille autorisée (1 Go) » — ou rien, selon le cas
Où la mettre	Dans le type, en constante `static final` nommée	Dans la configuration, et la contrôler au démarrage

L'erreur classique : une limite technique qui devient un message métier

Un service refuse un fichier de 64 Mo avec le message « Votre fichier est trop lourd pour nos serveurs ». C'est une limite technique déguisée en règle métier : elle changera au prochain passage à l'échelle, et le client s'en apercevra. À l'inverse, une limite métier non doublée d'une limite technique (pas de plafond de taille sur un import) transforme une saisie de 200 Mo en `OutOfMemoryError`, c'est-à-dire en panne. Les deux vont ensemble : **le métier décide, la technique protège**.

1.5 La grille de décision, en une page

Face à une entrée quelconque, quatre questions dans cet ordre suffisent à choisir le traitement. C'est la grille utilisée dans tout le document.

#	Question	Si la réponse est oui
1	La valeur peut-elle être convertie dans le type attendu ?	Sinon : erreur de **format** (champ, cause), on ne peut pas continuer sur ce champ (4.2)
2	Respecte-t-elle les bornes métier (min, max, longueur, format) ?	Sinon : erreur de **règle métier**, on peut accumuler les autres erreurs de la même saisie
3	Est-elle cohérente avec le reste (dates, état, quota, stock) ?	Sinon : erreur d'**état** — il faut lire le contexte, pas seulement le champ (3.5, 6.1, 8.2)
4	Le volume total reste-t-il sous le plafond technique ?	Sinon : refus **avant** de lire, pas après (5.3, 5.5)

Le test qui valide la grille

Prenez une entrée impossible de votre application et demandez-vous : à quelle question doit-elle échouer, et quel message l'utilisateur verra-t-il ? Si vous ne savez pas répondre, c'est que la limite n'est pas écrite — elle est seulement espérée. Le chapitre 11 liste les douze oublis les plus fréquents.

2 Les limites numériques

> Les nombres sont la première source de bugs silencieux, parce que Java ne signale presque jamais un dépassement : il calcule faux, et continue. Ce chapitre montre exactement où sont les murs, comment les rendre visibles, et pourquoi il ne faut jamais représenter de l'argent en `double`.

2.1 Les bornes des types entiers, et le débordement silencieux

Un `int` est codé sur 32 bits, un `long` sur 64. Le dépassement (overflow) ne lève aucune exception : les bits de poids fort sont simplement perdus, et le résultat peut devenir négatif. La JLS est explicite : « The integer operators do not indicate overflow or underflow in any way » (JLS §4.2.2).

PROGRAMME — LES BORNES, PUIS LE DÉPASSEMENT

```
System.out.println("Integer.MAX_VALUE = " + Integer.MAX_VALUE);
System.out.println("Integer.MIN_VALUE = " + Integer.MIN_VALUE);
System.out.println("Long.MAX_VALUE    = " + Long.MAX_VALUE);
System.out.println("MAX_VALUE + 1     = " + (Integer.MAX_VALUE + 1));

int stock = 2_000_000_000;
int ajout = 1_500_000_000;
System.out.println("stock + ajout     = " + (stock + ajout));
System.out.println("en long           = " + ((long) stock + ajout));
```

SORTIE RÉELLE

```
Integer.MAX_VALUE = 2147483647
Integer.MIN_VALUE = -2147483648
Long.MAX_VALUE    = 9223372036854775807
MAX_VALUE + 1     = -2147483648
stock + ajout     = -794967296
en long           = 3500000000
```

Notez le dernier point : le calcul est correct dès qu'un seul opérande est un `long`. C'est la promotion numérique qui décide du type de l'opération, pas votre intention. Deux réflexes en découlent : pour tout ce qui peut grandir (compteurs, stocks, totaux, identifiants), choisissez `long` par défaut ; et pour toute addition de deux valeurs venues de l'extérieur, utilisez la variante « exacte » ci-dessous.

2.2 `Math.*Exact` : rendre l'échec visible

Le JDK fournit depuis Java 8 une famille de méthodes dont le nom finit par `Exact` : elles lèvent une `ArithmeticException` au lieu de déborder. La Javadoc de `Math` le recommande explicitement : « In cases where the size is int or long and overflow errors need to be detected, the methods whose names end with Exact throw an ArithmeticException when the results overflow ».

PROGRAMME

```
try {
    System.out.println(Math.addExact(stock, ajout));
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException : " + e.getMessage());
}
try {
    System.out.println(Math.multiplyExact(1_000_000, 1_000_000));
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException : " + e.getMessage());
}
// conversion vérifiée d'un long vers un int
```

```
try {
    System.out.println(Math.toIntExact(5_000_000_000L));
} catch (ArithmeticException e) {
    System.out.println("toIntExact : " + e.getMessage());
}
```

SORTIE RÉELLE

```
ArithmeticException : integer overflow
ArithmeticException : integer overflow
toIntExact : integer overflow
```

Méthode	Quand l'utiliser
`Math.addExact`, `subtractExact`	Additionner un stock, un total, un compteur qui vient de l'extérieur.
`Math.multiplyExact`	Prix × quantité, surface × prix au m², points × multiplicateur de bonus.
`Math.incrementExact`, `decrementExact`	Compteurs d'essais, de tentatives, d'objets dans une partie.
`Math.negateExact`	Inverser un montant ou une quantité lue de l'extérieur.
`Math.toIntExact`	Repasser d'un `long` à un `int` (identifiant, index, taille) sans perdre de bits.

> **La règle simple**
>
> Dans le code qui traite une entrée, jamais `+`, `*` ou `-` « nus » sur des valeurs venues de l'extérieur : utilisez la version `Exact`. Dans une boucle chaude interne où les bornes sont prouvées, l'opérateur nu reste légitime — mais alors la preuve doit être écrite en commentaire.

2.3 Le piège de `Math.abs(Integer.MIN_VALUE)`

Parce que les entiers sont en complément à deux, la valeur absolue de `Integer.MIN_VALUE` n'est pas représentable dans un `int` : `Math.abs` renvoie donc… la même valeur négative. C'est un cas rare, mais c'est exactement le genre de valeur qu'une saisie peut fournir.

PROGRAMME

```
System.out.println("Math.abs(MIN_VALUE) = " + Math.abs(Integer.MIN_VALUE));
System.out.println("< 0 ?               = " + (Math.abs(Integer.MIN_VALUE) < 0));
try {
    System.out.println(Math.absExact(Integer.MIN_VALUE));
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException : " + e.getMessage());
}
```

SORTIE RÉELLE

```
Math.abs(MIN_VALUE) = -2147483648
< 0 ?               = true
ArithmeticException : Overflow to represent absolute value of Integer.MIN_VALUE
```

Conséquence pratique : un code du type `if (Math.abs(ecart) < seuil)` est faux pour une valeur sur quatre milliards, et cette valeur arrive toujours par une entrée. Utilisez `Math.absExact` (Java 15+) — ou comparez sans valeur absolue, ce qui est souvent plus simple : `if (ecart > -seuil && ecart < seuil)`.

2.4 Division, modulo et signes

La division entière par zéro est la seule opération arithmétique qui lève systématiquement une exception : `ArithmeticException` (JLS §15.17.2 pour la division, §15.17.3 pour le reste). Les autres cas — notamment les signes — produisent des résultats corrects mais souvent contre-intuitifs.

PROGRAMME

```
try {
    System.out.println(10 / 0);
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException : " + e.getMessage());
}
System.out.println("-7 / 2  = " + (-7 / 2) + "   -7 % 2 = " + (-7 % 2));
System.out.println("Math.floorDiv(-7, 2) = " + Math.floorDiv(-7, 2));
System.out.println("Math.floorMod(-7, 2) = " + Math.floorMod(-7, 2));
```

SORTIE RÉELLE

```
ArithmeticException : / by zero
-7 / 2  = -3   -7 % 2 = -1
Math.floorDiv(-7, 2) = -4
Math.floorMod(-7, 2) = 1
```

La division Java tronque vers zéro : pour un partage de butin, une pagination ou un calcul d'index, ce n'est pas ce qu'on veut. `Math.floorDiv` et `Math.floorMod` donnent le quotient et le reste « mathématiques », avec un reste toujours du signe du diviseur — la seule variante qui garantit `0 <= reste < diviseur`. C'est indispensable pour un index circulaire (un plateau de jeu qui revient au début) : `index = Math.floorMod(index, taille)`.

> **Le dénominateur nul dans un jeu ou un calcul de moyenne**
>
> `points / joueurs` lève `ArithmeticException` si `joueurs` vaut zéro — cas réel : une partie sans joueur, une moyenne sur une liste vide. Il n'y a pas de bonne réponse automatique : soit zéro est un état impossible et cette exception signale un vrai bug (bien), soit c'est un cas normal et il faut le traiter explicitement avant la division. Choisissez, et documentez.

2.5 Flottants : arrondi, Infinity, NaN

Un `double` ne représente pas exactement la plupart des décimaux. La conséquence n'est pas « une petite imprécision » : c'est une **comparaison fausse** et un **affichage faux**. Et pour les cas extrêmes, Java produit `Infinity` et `NaN` au lieu de lever une exception (JLS §15.18 pour les opérations arithmétiques flottantes).

PROGRAMME

```
System.out.println("0.1 + 0.2 = " + (0.1 + 0.2));
System.out.println("0.1+0.2 == 0.3 ? " + (0.1 + 0.2 == 0.3));
System.out.println("1e308 * 10 = " + (1e308 * 10));
System.out.println("0.0 / 0.0  = " + (0.0 / 0.0));
System.out.println("Double.NaN == Double.NaN : " + (Double.NaN == Double.NaN));
System.out.println("Double.isNaN(0.0/0.0) : " + Double.isNaN(0.0 / 0.0));
System.out.println("Double.parseDouble(\"1e999\") = " + Double.parseDouble("1e999"));
```

SORTIE RÉELLE

```
0.1 + 0.2 = 0.30000000000000004
0.1+0.2 == 0.3 ? false
1e308 * 10 = Infinity
0.0 / 0.0  = NaN
Double.NaN == Double.NaN : false
```

```
Double.isNaN(0.0/0.0) : true
Double.parseDouble("1e999") = Infinity
```

Symptôme	Cause	Correctif
`0.1 + 0.2 != 0.3`	Représentation binaire des décimaux	Comparer avec une tolérance, ou utiliser `BigDecimal` pour les sommes d'argent
`Infinity` dans un total	Débordement de `double` : silencieux	Tester `Double.isInfinite` après le calcul, ou passer en `BigDecimal`
`NaN` qui contamine tout	`0.0/0.0`, ou une opération sur `NaN`	Tester `Double.isNaN` ; ne jamais comparer avec `==`
`NaN` en fin de tri	`Double.compare` place `NaN` après tout	Filtrer avant de trier : `filter(Double::isFinite)`
« 0.3 » à l'affichage d'un `float`	`Float.toString` produit la chaîne la plus courte qui identifie le `float`	Le `float` vaut 0.30000001192092896 en `double` : ne pas s'en servir pour un montant

> **Le `float` qui « a l'air juste »**
>
> `0.1f + 0.2f` affiche `0.3`. Ce n'est pas une preuve d'exactitude : c'est la conversion en chaîne la plus courte qui identifie le `float` résultat. Converti en `double`, il vaut `0.30000001192092896`. Autrement dit, l'affichage cache l'erreur — exactement ce qu'on veut éviter dans un document sur la vérification des entrées.

2.6 L'argent : centimes entiers ou `BigDecimal`

Deux représentations sont acceptables, une seule est dangereuse :

Représentation	Quand	Ce qu'il faut retenir
`long` en centimes	Monnaie unique, calculs simples, stockage en base	Exact, rapide, sérialisable sans surprise. `1999` = 19,99 €. Attention à la conversion à l'affichage seulement.
`BigDecimal`	Plusieurs devises, taux, arrondis réglementaires	Exact en décimal, mais **immuable** et plus lent. Il faut toujours préciser l'échelle et le mode d'arrondi.
`double` / `float`	Jamais pour de l'argent	Impossible à arrondir de façon fiable. Les erreurs d'un centime se cumulent et se voient en comptabilité.

PROGRAMME — LES DEUX BONNES FAÇONS, ET LE PIÈGE DU CONSTRUCTEUR

```
long prixCentimes = 1999;
System.out.println("1999 centimes x3 = " + (prixCentimes * 3) + " centimes");

BigDecimal prix = new BigDecimal("19.99");          // String : exact
System.out.println("19.99 x 3        = " + prix.multiply(BigDecimal.valueOf(3)));
System.out.println("TVA 20 %         = " + prix.multiply(new BigDecimal("0.20")).setScale(2,
RoundingMode.HALF_UP));

System.out.println("new BigDecimal(0.1)     = " + new BigDecimal(0.1));
System.out.println("BigDecimal.valueOf(0.1) = " + BigDecimal.valueOf(0.1));
```

SORTIE RÉELLE

```
1999 centimes x3 = 5997 centimes
19.99 x 3        = 59.97
TVA 20 %         = 4.00
new BigDecimal(0.1)     = 0.1000000000000000055511151231257827021181583404541015625
BigDecimal.valueOf(0.1) = 0.1
```

La dernière ligne mérite d'être relue deux fois. `new BigDecimal(0.1)` prend le `double` tel qu'il est réellement stocké — la Javadoc du constructeur dit « the exact decimal representation of the double's binary floating-point value » — et vous obtenez 55 décimales. `BigDecimal.valueOf(0.1)` passe par `Double.toString` et donne `0.1`. Pour construire depuis une saisie, utilisez le constructeur `String` ; pour convertir un `double` existant, utilisez `valueOf`.

PROGRAMME — LES TROIS ERREURS DE `BIGDECIMAL` QUI FONT ÉCHOUER UNE PRODUCTION

```
try { System.out.println(BigDecimal.ONE.divide(new BigDecimal("3"))); }
catch (ArithmeticException e) { System.out.println("divide sans échelle -> " + e.getMessage()); }

System.out.println("divide(3, 2, HALF_UP) = " + BigDecimal.ONE.divide(new BigDecimal("3"), 2,
RoundingMode.HALF_UP));
System.out.println("setScale(2) de 2.345  = " + new BigDecimal("2.345").setScale(2,
RoundingMode.HALF_UP));
try { System.out.println(new BigDecimal("2.35").setScale(1)); }
catch (ArithmeticException e) { System.out.println("setScale(1) de 2.35 -> " + e.getMessage()); }
try { new BigDecimal("12,50"); }
catch (NumberFormatException e) { System.out.println("BigDecimal(\"12,50\") -> " +
e.getMessage()); }
```

SORTIE RÉELLE

```
divide sans échelle -> Non-terminating decimal expansion; no exact representable decimal result.
divide(3, 2, HALF_UP) = 0.33
setScale(2) de 2.345  = 2.35
setScale(1) de 2.35 -> Rounding necessary
BigDecimal("12,50") -> Character , is neither a decimal digit number, decimal point, nor "e" notation
exponential mark.
```

Trois règles, donc : **toute division précise son échelle et son arrondi** (`divide(x, 2, RoundingMode.HALF_UP)`) ; **toute réduction d'échelle précise l'arrondi** (`setScale(2, HALF_UP)`), sinon `Rounding necessary` ; et **la virgule française n'est pas un séparateur décimal** — il faut normaliser la saisie avant de construire le `BigDecimal` (chapitre 9.4).

2.7 Convertir une saisie : `parseInt` et ses pièges

`Integer.parseInt` est strict : la Javadoc précise que tous les caractères doivent être des chiffres décimaux, à l'exception éventuelle d'un signe en tête, et qu'une `NumberFormatException` est levée si la chaîne est `null`, vide, contient un caractère non numérique ou dépasse la capacité d'un `int`. Aucun espace n'est toléré, aucune virgule, aucun `+` ailleurs qu'en tête.

PROGRAMME — SEPT SAISIES, SEPT RÉPONSES

```
for (String s : new String[] { "42", " 42", "4 2", "", "42.0", "99999999999", null }) {
    try {
        System.out.println("\"" + s + "\" -> " + Integer.parseInt(s));
    } catch (NumberFormatException e) {
        System.out.println("\"" + s + "\" -> NumberFormatException : " + e.getMessage());
    }
}
```

SORTIE RÉELLE

```
"42"        -> 42
" 42"       -> NumberFormatException : For input string: " 42"
"4 2"       -> NumberFormatException : For input string: "4 2"
""          -> NumberFormatException : For input string: ""
"42.0"      -> NumberFormatException : For input string: "42.0"
"99999999999" -> NumberFormatException : For input string: "99999999999"
"null"      -> NumberFormatException : Cannot parse null string
```

La chaîne de conversion correcte d'un entier saisi

1. `strip()` la chaîne (les espaces de saisie sont normaux, on les enlève sciemment). **2.** `parseInt` — et si le format est bon mais la valeur hors des bornes du type, vous obtenez aussi une `NumberFormatException` : le message ne distingue pas les deux cas, c'est à vous de les séparer. **3.** Vérifier les bornes métier (1 à 5 articles) séparément. **4.** Pour un message utile, ne renvoyez jamais le message du JDK à l'utilisateur : « For input string: "4 2" » ne dit rien ; « le nombre d'articles doit être un entier entre 1 et 5 » dit tout.

PROGRAMME — CONVERSION SANS EXCEPTION, QUAND L'ABSENCE DE VALEUR EST NORMALE

```
static OptionalInt parse(String s) {
    try {
        return OptionalInt.of(Integer.parseInt(s.strip()));
    } catch (NumberFormatException e) {
        return OptionalInt.empty();
    }
}
```

SORTIE RÉELLE

```
parse(" 42 ")         = OptionalInt[42]
parse("abc")         = OptionalInt.empty
parse("99999999999") = OptionalInt.empty
```

Cette variante est à préférer quand l'échec est une issue normale (un champ facultatif, un filtre de recherche) et non une erreur à signaler. La section 7.1 donne le critère exact pour choisir entre `Optional` et exception.

3 Les limites de dates et de durées

> « 5 jours maximum » paraît simple. C'est en réalité le cas le plus piégeux du document : cinq jours ne sont pas 120 heures, un mois ne fait pas 30 jours, le 30 février est accepté par certains parseurs, et sans horloge injectée votre règle « la date ne peut pas être passée » ne se teste pas. Ce chapitre fixe les définitions une fois pour toutes.

3.1 `Clock` : le temps doit être injectable

Première règle, et la plus souvent ignorée : **un code qui lit l'heure n'est pas testable**. `LocalDate.now()` rend un résultat différent demain : impossible de tester « la date est déjà passée » sans attendre 24 heures. La solution du JDK est `java.time.Clock` : on lit le temps à travers une horloge fournie de l'extérieur.

PROGRAMME — LA MÊME RÈGLE, AVEC UNE HORLOGE FIGÉE

```
Clock fixe = Clock.fixed(Instant.parse("2026-11-10T09:00:00Z"), ZoneId.of("Europe/Paris"));
LocalDate aujourdhui = LocalDate.now(fixe);
System.out.println("aujourd'hui (horloge fixe) = " + aujourdhui);
```

SORTIE RÉELLE

```
aujourd'hui (horloge fixe) = 2026-11-10
```

En production, on injecte `Clock.systemDefaultZone()` (ou `Clock.systemUTC()`) ; en test, `Clock.fixed(...)`. La signature typique : `DemandeLocation valider(Saisie saisie, Clock horloge)`. C'est ce que fait le service du chapitre 4, et c'est ce qui permet aux tests de vieillir sans casser.

3.2 Compter des jours : inclusif, exclusif, calendaire

« Du 12 au 16 novembre, c'est combien de jours ? » Quatre réponses différentes selon la définition. C'est la source d'erreur numéro un des réservations et des plannings.

Définition	Calcul	Usage
Écart (exclusif)	`ChronoUnit.DAYS.between(d1, d2)`	Nombre de jours entre deux instants. 12 → 16 donne **4**.
Durée calendaire (inclusive)	`between(...) + 1`	Nombre de jours occupés. 12 → 16 donne **5**. C'est la bonne définition pour « 5 jours de location ».
Nuits	`between(...)` sans `+1`	Hôtellerie : une arrivée le 12 et un départ le 16, c'est 4 nuits.
Jours ouvrés	Boucle ou `ChronoUnit.DAYS` + test du jour de la semaine	Délais de traitement : il faut exclure samedis, dimanches et jours fériés (aucune méthode du JDK ne le fait seule).

PROGRAMME — LES DEUX CHIFFRES, CÔTE À CÔTE

```
LocalDate debut = LocalDate.of(2026, 11, 10);
LocalDate fin = debut.plusDays(5);
System.out.println("debut=" + debut + " fin=" + fin + " ecart=" + ChronoUnit.DAYS.between(debut, fin));
System.out.println("durée inclusive = " + (ChronoUnit.DAYS.between(debut, fin) + 1) + " jours");
```

SORTIE RÉELLE

```
debut=2026-11-10 fin=2026-11-15 ecart=5
durée inclusive = 6 jours calendaires
```

L'erreur d'un jour, garantie

Un service qui applique « maximum 5 jours » avec `ChronoUnit.DAYS.between(debut, fin) > 5` accepte en réalité une location de **6 jours** (du 10 au 15 inclus). L'inverse — ajouter `+1` partout — refuse une location de 5 jours valides. La seule parade : écrire la définition dans le nom de la méthode (`nbJoursCalendaires()`, `nbNuits()`) et la tester avec une date de début et une date de fin réelles.

3.3 `plusDays(1)` n'est pas 24 heures

Un jour civil dure 24 heures... sauf deux fois par an, dans les fuseaux qui changent d'heure. `Duration` compte des heures réelles ; `Period` compte des unités calendaires. Choisir le mauvais type produit une heure de décalage, deux fois par an, sur les réservations.

PROGRAMME — 24 HEURES CONTRE 1 JOUR

```
ZoneId paris = ZoneId.of("Europe/Paris");
ZonedDateTime avant = ZonedDateTime.of(2026, 3, 28, 12, 0, 0, 0, paris);
ZonedDateTime apres = avant.plusDays(1);
System.out.println("28/03 12:00 + 24 h (Duration) = " + avant.plus(Duration.ofHours(24)));
System.out.println("28/03 12:00 + 1 jour (Period) = " + apres);
System.out.println("heures réelles écoulées       = " + Duration.between(avant, apres).toHours());
```

SORTIE RÉELLE

```
28/03 12:00 + 24 h (Duration) = 2026-03-29T13:00+02:00[Europe/Paris]
28/03 12:00 + 1 jour (Period) = 2026-03-29T12:00+02:00[Europe/Paris]
heures réelles écoulées       = 23
```

Le 29 mars 2026, la journée dure 23 heures à Paris (passage à l'heure d'été). `plusDays(1)` conserve l'heure locale — c'est presque toujours ce qu'on veut pour une règle métier. `Duration.ofHours(24)` avance d'un jour réel, donc décale l'heure locale d'une heure. Règle : **dates et plannings → `Period` / `ChronoUnit` ; durées mesurées, délais techniques, temps de traitement → `Duration`**.

PROGRAMME — AJOUTER UN MOIS N'EST PAS AJOUTER 30 JOURS

```
LocalDate d1 = LocalDate.of(2026, 1, 31);
System.out.println("31/01 + 1 mois = " + d1.plusMonths(1));
System.out.println("Period.between(31/01, 28/02) = " + Period.between(d1, LocalDate.of(2026, 2,
28)));
```

SORTIE RÉELLE

```
31/01 + 1 mois = 2026-02-28
Period.between(31/01, 28/02) = P28D
```

3.4 Les formats d'entrée : `uuuu`, `ResolverStyle`, et le 30 février

C'est ici que se joue la différence entre « une date qui ressemble à une date » et « une date qui existe ». `DateTimeFormatter` a un comportement par défaut plus permissif qu'on ne le croit.

PROGRAMME — CINQ SAISIES AU FORMAT ISO

```
DateTimeFormatter iso = DateTimeFormatter.ISO_LOCAL_DATE;
for (String s : new String[] { "2026-11-10", "10/11/2026", "2026-02-30", "2026-13-01",
"2026-11-10T10:00" }) {
    try {
        System.out.println(s + " -> " + LocalDate.parse(s, iso));
```

```
    } catch (DateTimeParseException e) {
        System.out.println(s + " -> DateTimeParseException : " + e.getMessage());
    }
}
```

SORTIE RÉELLE

```
2026-11-10          -> 2026-11-10
10/11/2026          -> DateTimeParseException : Text '10/11/2026' could not be parsed at index 0
2026-02-30          -> DateTimeParseException : Text '2026-02-30' could not be parsed: Invalid date
'FEBRUARY 30'
2026-13-01          -> DateTimeParseException : Text '2026-13-01' could not be parsed: Invalid value
for MonthOfYear (valid values 1 - 12): 13
2026-11-10T10:00    -> DateTimeParseException : Text '2026-11-10T10:00' could not be parsed, unparsed
text found at index 10
```

Bonne nouvelle : le parseur ISO est strict sur les dates impossibles. Mauvaise nouvelle : un formateur « maison » ne l'est pas par défaut. Le `ResolverStyle` par défaut d'un formateur construit par `ofPattern` est **SMART**, pas `STRICT` — et SMART corrige les dates invalides.

PROGRAMME — LE 30 FÉVRIER DEVIENT LE 28 FÉVRIER, SANS ERREUR

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/uuuu", Locale.FRANCE);
System.out.println("RESOLVER strict par défaut ? " + fr.getResolverStyle());

DateTimeFormatter smart = DateTimeFormatter.ofPattern("dd/MM/
uuuu").withResolverStyle(ResolverStyle.SMART);
System.out.println("SMART  30/02/2026 -> " + LocalDate.parse("30/02/2026", smart));

DateTimeFormatter strict = DateTimeFormatter.ofPattern("dd/MM/
uuuu").withResolverStyle(ResolverStyle.STRICT);
try {
    System.out.println(strict.parse("30/02/2026"));
} catch (DateTimeParseException e) {
    System.out.println("STRICT 30/02/2026 -> refusée : " + e.getMessage().split("\n")[0]);
}
```

SORTIE RÉELLE

```
RESOLVER strict par défaut ? SMART
SMART  30/02/2026 -> 2026-02-28
STRICT 30/02/2026 -> refusée : Text '30/02/2026' could not be parsed: Invalid date 'FEBRUARY 30'
```

> **La règle des deux `u`**
>
> Dans un motif de date, `uuuu` désigne l'**année proleptique** (celle qu'on attend : 2026), tandis que `yyyy` désigne l'année d'ère, qui n'existe qu'accompagnée d'une ère. Avec `yyyy` et un résolveur `STRICT`, la conversion échoue toujours : le parseur obtient bien une année d'ère, mais ne sait pas en déduire une date — `Unable to obtain LocalDate from TemporalAccessor: {MonthOfYear=11, YearOfEra=2026, DayOfMonth=10}`. Avec `uuuu`, elle est sans ambiguïté, donc utilisez toujours `uuuu` — et ajoutez `.withResolverStyle(ResolverStyle.STRICT)` dès que la date vient de l'extérieur.

Besoins	Formateur recommandé
Date ISO d'une API	`DateTimeFormatter.ISO_LOCAL_DATE` (strict, pas de réglage)
Date saisie par un utilisateur français	`ofPattern("dd/MM/uuuu", Locale.FRANCE).withResolverStyle(STRICT)`
Date + heure avec fuseau	`DateTimeFormatter.ISO_ZONED_DATE_TIME`, ou `ISO_OFFSET_DATE_TIME` si le fuseau n'est pas nécessaire
Affichage d'un montant ou d'une date	`ofLocalizedDate(FormatStyle.LONG)` avec une `Locale` explicite, jamais `Locale.getDefault()`

3.5 Plages, chevauchements et bornes du temps

Une plage de dates est un excellent exemple d'objet à invariants : début obligatoire, fin après début, durée sous un plafond. Une fois ces trois contrôles dans le constructeur, plus aucune méthode de l'application ne peut travailler sur une plage absurde.

PROGRAMME — LA PLAGE QUI VALIDE SA PROPRE EXISTENCE

```
record Plage(LocalDate debut, LocalDate fin) {
    static final int JOURS_MAX = 5;

    Plage {
        if (debut == null || fin == null) throw new IllegalArgumentException("dates
obligatoires");
        if (fin.isBefore(debut)) throw new IllegalArgumentException("la fin (" + fin + ") précède
le début (" + debut + ")");
        long jours = ChronoUnit.DAYS.between(debut, fin) + 1;
        if (jours > JOURS_MAX) throw new IllegalArgumentException("plage de " + jours + " jours :
maximum " + JOURS_MAX);
    }

    long nbJours() { return ChronoUnit.DAYS.between(debut, fin) + 1; }

    boolean chevauche(Plage autre) {
        return !fin.isBefore(autre.debut) && !autre.fin.isBefore(debut);
    }
}
```

SORTIE RÉELLE

```
Plage[debut=2026-07-04, fin=2026-07-08] dure 5 jours (maximum 5)
refus : plage de 48 jours : maximum 5
refus : la fin (2026-07-04) précède le début (2026-08-20)
refus : plage de 6 jours : maximum 5
refus : dates obligatoires
chevauche ? true
chevauche ? false
```

Le test de chevauchement mérite d'être retenu tel quel : deux plages se chevauchent si et seulement si la fin de l'une n'est pas avant le début de l'autre dans les deux sens. La condition écrite avec les négations est celle qui gère correctement les plages qui se touchent (fin = début) — à vous de décider si c'est un chevauchement, et de le documenter.

PROGRAMME — LES BORNES DU TEMPS EXISTENT AUSSI

```
System.out.println("LocalDate.MAX = " + LocalDate.MAX);
try { LocalDate.MAX.plusDays(1); }
catch (DateTimeException e) { System.out.println("  " + e.getMessage()); }

try { Instant.MAX.plusSeconds(1); }
```

```
catch (DateTimeException e) { System.out.println("  " + e.getMessage()); }

try { LocalDate.of(2026, 2, 30); }
catch (DateTimeException e) { System.out.println("LocalDate.of(2026,2,30) -> " +
e.getMessage()); }
```

SORTIE RÉELLE

```
LocalDate.MAX = +999999999-12-31
  Invalid value for Year (valid values -999999999 - 999999999): 1000000000
  Instant exceeds minimum or maximum instant
LocalDate.of(2026,2,30) -> Invalid date 'FEBRUARY 30'
```

Pourquoi ces bornes arrivent en production

Une date de fin calculée à partir d'une saisie (`debut.plusDays(n)`) peut sortir de l'intervalle si `n` vient de l'extérieur. Et un export mal formé peut contenir `9999-12-31`, utilisé comme « date infinie » : à laquelle on ajoute un jour. Ces trois exceptions sont toutes des `DateTimeException` — inutile de les différencier pour l'utilisateur, mais il faut les laisser remonter au lieu de renvoyer `null`.

4 Structurer la validation d'entrée

> Vous savez maintenant quelles limites existent. Reste la question d'ingénierie : où placer les contrôles, dans quel ordre, et comment rendre les erreurs utilisables. Ce chapitre construit le convertisseur complet de la demande de location — celui qui refusera une saisie de 9 jours et de 6 articles en expliquant les deux.

4.1 Valider et convertir : deux verbes différents

Un convertisseur fait trois choses, dans cet ordre, et il ne faut pas les mélanger :

Étape	Ce qu'elle fait	En cas d'échec
1. Convertir	Transformer le texte en type cible : `String` → `int`, `LocalDate`, `BigDecimal`	Erreur de forme : le champ ne peut pas être interprété. Souvent bloquant pour ce champ seul.
2. Valider	Vérifier des règles sur la valeur convertie : bornes, format, longueur, cohérence	Erreur de règle : la valeur est compréhensible mais interdite. Non bloquante : on continue.
3. Construire	Appeler le constructeur, qui revérifie les invariants (ceinture et bretelles, mais gratuites)	Erreur de programmation : le convertisseur a laissé passer quelque chose. Bug à corriger, pas à afficher.

> **Un convertisseur, un seul**
>
> Le formulaire, la ligne de commande, le fichier importé et la requête HTTP doivent tous passer par le même `depuis…()`. Sinon vous aurez quatre jeux de règles qui divergent en six mois. C'est la raison d'être du chapitre : la validation n'est pas une propriété des contrôleurs, c'est une propriété du domaine.

4.2 Accumuler les erreurs ou s'arrêter à la première

Deux stratégies, deux usages, et il faut savoir choisir consciemment.

Stratégie	Avantage	Inconvénient
Premier échec (exception immédiate)	Code court, une seule chose à corriger, la cause exacte est conservée. Idéal pour une API machine-à-machine ou une ligne de commande.	L'utilisateur corrige une erreur, en découvre une autre, recommence : 5 allers-retours pour 5 erreurs.
Accumulation (liste de violations)	Un seul aller-retour, tous les champs fautifs signalés ensemble. Idéal pour un formulaire humain.	Code plus lourd, il faut un type de résultat et une règle de plafonnement (ne pas renvoyer 500 erreurs).

PROGRAMME — LA MÊME SAISIE, ENTIÈREMENT FAUSSE, VUE PAR LES DEUX STRATÉGIES

```
// accumulation : on signale les cinq champs fautifs d'un coup
try {
    DemandeLocation.depuisFormulaire(AUJ, "loc 2026", "2026-11-12", "9", "0", "12",
"x".repeat(201));
} catch (ErreurValidation e) {
    for (ErreurValidation.Violation v : e.violations()) {
        System.out.println("  - champ=" + v.champ() + " code=" + v.code() + " : " + v.message());
```

```
    }
}

// premier échec : on s'arrête au premier champ illisible
try {
    DemandeLocation.depuisLigne("LOC-1;2026-11-12;cinq;1;2");
} catch (ChampInvalideException e) {
    System.out.println("champ=" + e.champ() + " message=" + e.getMessage() + " cause=" +
e.getCause().getClass().getSimpleName());
}
```

SORTIE RÉELLE

```
  - champ=reference code=FORMAT_INVALIDE : 3 à 20 caractères parmi A-Z, 0-9 et tiret : loc 2026
  - champ=jours code=VALEUR_HORS_BORNES : valeur attendue entre 1 et 5, reçu 9
  - champ=articles code=VALEUR_HORS_BORNES : valeur attendue entre 1 et 5, reçu 0
  - champ=passagers code=VALEUR_HORS_BORNES : valeur attendue entre 1 et 9, reçu 12
  - champ=commentaire code=VALEUR_HORS_BORNES : 200 caractères maximum, reçu 201
champ=jours message=nombre entier attendu : "cinq" cause=NumberFormatException
```

Remarquez la `cause` sur la dernière ligne : même en stratégie « premier échec », on n'écrase pas l'exception d'origine (`NumberFormatException`). C'est cette information qui permet de distinguer un bug de conversion d'une saisie fautive quand on lit les journaux.

4.3 Le résultat de validation et ses quatre issues

Une opération de conversion peut se terminer de quatre façons. Les confondre est la source de la moitié des messages d'erreur inutiles.

Issue	Exemple	Représentation Java
Succès	saisie valide	L'objet, ou `Optional.of(obj)` si l'absence est normale
Rejet	6 articles au lieu de 5	`ErreurValidation` (liste de violations) — c'est un résultat, pas une panne
Conflit d'état	quota dépassé, stock insuffisant, date déjà réservée	Exception métier typée (`QuotaDepasseException`…), levée après la validation des champs
Panne	base de données injoignable, fichier de configuration absent	Exception technique enveloppée, cause conservée, message technique pour les journaux

> **Ne mélangez pas « rejet » et « panne »**
>
> Un rejet est une réponse normale du système à une demande incorrecte : il doit être rapide, complet et affichable. Une panne est un incident : elle doit être journalisée avec sa cause et ne jamais afficher au client un message interne. Le test simple : si le message peut être affiché tel quel à un utilisateur sans l'inquiéter, c'est un rejet ; sinon, c'est une panne.

PROGRAMME — LE TYPE DE RÉSULTAT, UTILISÉ PAR L'APPELANT

```
// conversion « silencieuse » : on veut juste savoir si ça passe
System.out.println(DemandeLocation.essayer(new DemandeLocation.Saisie(
        AUJ, "LOC-2026-11", "2026-11-12", "5", "5", "4", null)).map(DemandeLocation::fin));
System.out.println(DemandeLocation.essayer(new DemandeLocation.Saisie(
        AUJ, "LOC-2026-11", "2026-11-12", "8", "5", "4", null)));
```

```
// comptage : combien de violations pour une saisie ?
try { DemandeLocation.depuisFormulaire(AUJ, "X", "2020-01-01", "-1", "0", "99", null); }
catch (ErreurValidation e) { System.out.println(e.violations().size() + " violations"); }
```

SORTIE RÉELLE

```
Optional[2026-11-16]
Optional.empty
5 violations
```

4.4 Les six limites de saisie et leurs messages

Retenez ce tableau : six familles de limites couvrent presque tous les champs d'un formulaire professionnel. Pour chacune, la colonne de droite indique le message type — court, sans jargon, avec la limite et la valeur reçue.

Limite	Vérification	Cas piégeux	Message type
Présence	`s == null || s.isBlank()`	`""`, `" "`, `"\u00a0"` (espace insécable)	« La référence est obligatoire. »
Longueur	`s.length() <= max`	Comptage en caractères UTF-16 : un emoji vaut 2	« 200 caractères maximum, reçu 201. »
Format	`s.matches("[A-Z0-9-]{3,20}")`	Accents, espaces, caractères invisibles	« 3 à 20 caractères parmi A-Z, 0-9 et tiret. »
Bornes numériques	`v >= min && v <= max`	Négatif, zéro, dépassement de type géré par `parseInt`	« Valeur attendue entre 1 et 5, reçu 9. »
Cohérence interne	fin ≥ début, passagers ≥ conducteur, total = somme	Champs liés : à valider ensemble, pas séparément	« La fin doit suivre le début. »
Domaine	date non passée, quota, stock, référence unique	Nécessite l'horloge ou un accès aux données (hors constructeur)	« Cette date est déjà passée : 2026-11-01. »

> **Compter des caractères : `length()` n'est pas ce que vous croyez**
>
> `String.length()` renvoie un nombre d'unités UTF-16, pas de caractères visibles. `"Café"` fait 4 unités et 4 points de code, mais un caractère hors BMP (un emoji, écrit `"\uD83D\uDC4D"`) fait **2 unités et 1 point de code** — vérifié au chapitre 11.4 ; « e◌́ » (e + accent combinant) fait 2 unités et 2 points de code mais s'affiche comme un seul caractère. Pour une limite de longueur destinée à un stockage, utilisez `length()` (c'est ce que la colonne `VARCHAR` compte). Pour une limite visuelle, comptez les points de code : `s.codePointCount(0, s.length())`, après normalisation en NFC (chapitre 9.4).

Atelier — 20 min objectif : écrire un convertisseur qui accumule les erreurs

☐ Écrivez `record Reservation(LocalDate debut, int jours, int articles)` avec un constructeur compact qui impose 1 ≤ jours ≤ 5 et 1 ≤ articles ≤ 5.

☐ Ajoutez `static Reservation depuisSaisie(String debut, String jours, String articles, Clock horloge)` qui accumule les erreurs dans une liste de violations (champ, code, message).

☐ Testez avec une saisie entièrement vide : vous devez obtenir trois violations, pas une exception.

☐ Testez avec `debut = "2026-02-30"` : vérifiez que le code renvoyé est `DATE_INCOHERENTE` et non `FORMAT_INVALIDE` — la distinction vient du fait que la chaîne a le bon format mais pas la bonne valeur.

Critère de réussite : aucune des deux méthodes `depuis…()` ne lève d'`IllegalArgumentException` ; seules les exceptions du type « validation » sortent, et chaque violation porte le nom du champ.

4.5 Où placer chaque contrôle : le tableau de décision

La question revient à chaque nouveau champ ajouté à un formulaire. Ce tableau donne la réponse en une ligne, et la raison : il n'y a que cinq emplacements possibles, et deux seulement sont obligatoires.

Emplacement	Ce qu'on y met	Pourquoi / pourquoi pas ailleurs
Contrôleur, interface, CLI	Conversion du texte vers les types, messages destinés à l'écran	Seul endroit où l'on sait « d'où vient » la donnée (HTTP, clavier, fichier). Ne doit contenir **aucune** règle métier.
Convertisseur `depuis…()`	Conversion + contrôle de forme + accumulation des violations	Point de passage unique : c'est lui qui garantit que le constructeur ne lèvera pas.
Constructeur du type	Invariants : bornes, cohérence interne, non-nullité	Obligatoire. C'est la seule protection qui vaut pour tous les chemins d'appel, y compris ceux qui n'existent pas encore.
Service / domaine	Règles qui dépendent du contexte : horloge, quota, stock, unicité	Impossible dans le constructeur : il faudrait y injecter l'horloge et la base. À faire après la construction, avant l'effet.
Base de données	Dernier filet : `NOT NULL`, contraintes d'unicité, types	Ne remplace rien. Une contrainte en base protège l'intégrité, pas l'expérience utilisateur : son message d'erreur est illisible.

Le test qui vérifie que vous êtes au bon endroit

Supprimez mentalement le contrôleur. Si les invariants tiennent encore (le type refuse de se construire faux), votre conception est correcte. Si l'application accepte une valeur impossible dès qu'on l'appelle depuis une tâche planifiée, votre validation est au mauvais endroit — elle est dans un contrôleur, pas dans le domaine.

5 Volumétrie : compter les objets, les octets, la profondeur

> Une limite de 5 objets est une règle métier. Une limite de 64 Mo est une règle technique. Entre les deux, il y a une zone grise où l'on croit protéger son application alors qu'on ne protège rien : collections non bornées, chaînes construites en boucle, JSON imbriqué, fichiers lus entièrement en mémoire. Ce chapitre borne les trois ressources : le nombre d'éléments, la mémoire, la profondeur.

5.1 Collections : plafond métier et plafond technique

Une collection Java n'a pas de taille maximale « métier » : elle grandit jusqu'à la mémoire disponible. Le seul plafond réel est la taille d'un tableau, plafonnée à `Integer.MAX_VALUE - 8` éléments — une constante présente dans les JDK pour laisser de la place à l'en-tête d'objet :

PROGRAMME — LE PLAFOND DU LANGAGE, PUIS LES EXCEPTIONS RÉELLES

```
System.out.println("Integer.MAX_VALUE - 8 = " + (Integer.MAX_VALUE - 8));

List<Integer> nombres = List.of(10, 20, 30, 40, 50);
System.out.println("subList(1,4) = " + nombres.subList(1, 4));
try { nombres.subList(3, 9); }
catch (IndexOutOfBoundsException e) { System.out.println("subList(3,9) -> " + e.getMessage()); }
try { nombres.get(-1); }
catch (IndexOutOfBoundsException e) { System.out.println("get(-1) -> " + e.getMessage()); }
```

SORTIE RÉELLE

```
Integer.MAX_VALUE - 8 = 2147483639
subList(1,4) = [20, 30, 40]
subList(3,9) -> IndexOutOfBoundsException : toIndex = 9
get(-1) -> IndexOutOfBoundsException : Index -1 out of bounds for length 5
```

Deux milliards d'éléments, c'est le plafond théorique ; en pratique, votre tas vous arrêtera bien avant (section 5.3). Le plafond qui vous intéresse est donc **métier** : 5 articles par commande, 9 passagers par véhicule, 3 commandes par tour. Il s'écrit dans le type, comme au chapitre 1 :

PROGRAMME — REFUSER 0 ET 6, ACCEPTER 1 À 5

```
static final int ARTICLES_MAX = 5;

static int accepter(int n) {
    if (n < 1) throw new IllegalArgumentException("nombre d'articles minimal : 1, reçu " + n);
    if (n > ARTICLES_MAX) throw new IllegalArgumentException("nombre d'articles maximal : " +
ARTICLES_MAX + ", reçu " + n);
    return n;
}
```

SORTIE RÉELLE

```
  accepter(0) -> refus : nombre d'articles minimal : 1, reçu 0
  accepter(1) -> 1
  accepter(5) -> 5
  accepter(6) -> refus : nombre d'articles maximal : 5, reçu 6
  accepter(-1) -> refus : nombre d'articles minimal : 1, reçu -1
```

> **Le plafond qui n'existe que dans le code d'appel**
>
> Un `if (articles > 5) return erreur;` dans le contrôleur, c'est zéro protection pour la méthode de service appelée ailleurs (tâche planifiée, import, test d'intégration). La limite doit être portée par le type, et la constante doit être `public static final` pour que le contrôleur affiche la même valeur que celle vérifiée — plutôt qu'un `5` recopié qui divergera.

5.2 Listes immuables, `null` interdit, fuites évitées

Quand un objet validé contient une collection, deux dangers apparaissent : l'appelant peut la modifier après coup (la validation ne vaut plus rien), et elle peut contenir `null`. Les fabriques immuables du JDK sont faites pour ça.

PROGRAMME — CE QUE LES FABRIQUES IMMUABLES REFUSENT, ET CE QU'ELLES PROTÈGENT

```
List<String> mutable = new ArrayList<>(List.of("a", "b"));
List<String> copie = List.copyOf(mutable);
mutable.add("c");
System.out.println("mutable=" + mutable + " copie=" + copie);
try { copie.add("d"); }
catch (UnsupportedOperationException e) { System.out.println("copie.add ->
UnsupportedOperationException"); }

try { List.of(1, null, 3); }
catch (NullPointerException e) { System.out.println("List.of(1, null, 3) ->
NullPointerException"); }
try { List.copyOf(Arrays.asList("a", null)); }
catch (NullPointerException e) { System.out.println("List.copyOf(avec null) ->
NullPointerException"); }

List<String> vue = Arrays.asList("a", "b");
try { vue.add("c"); }
catch (UnsupportedOperationException e) { System.out.println("Arrays.asList(...).add ->
UnsupportedOperationException"); }
vue.set(0, "z");
System.out.println("Arrays.asList(...).set(0) -> " + vue);
```

SORTIE RÉELLE

```
mutable=[a, b, c] copie=[a, b]
copie.add -> UnsupportedOperationException
List.of(1, null, 3) -> NullPointerException
List.copyOf(avec null) -> NullPointerException
Arrays.asList(...).add -> UnsupportedOperationException (taille figée)
Arrays.asList(...).set(0, "z") -> [z, b]
```

Fabrique	Taille	À utiliser pour
`List.of(...)`, `Set.of(...)`, `Map.of(...)`	Illimitée (varargs au-delà de 10)	Un objet validé qui contient une collection **finie et connue**. Refuse `null`.
`List.copyOf(coll)`, `Set.copyOf`, `Map.copyOf`	Celle de la source	Copier une entrée validée : la copie ne bouge plus, et la source peut être jetée. Refuse `null`.
`Arrays.asList(tab)`	Figée	Adapter un tableau existant. Attention : vue, donc `set` écrit dans le tableau d'origine.
`Collections.unmodifiableList(coll)`	Celle de la source	Vue non modifiable, mais la source reste modifiable : insuffisant pour figer un objet validé.
`new ArrayList<>(...)`	Libre	Collections de travail internes. Accepte `null` : à ne jamais exposer tel quel.

> **La règle en une ligne**
>
> Dans un objet validé : `List.copyOf(...)` à la construction, et retour du même objet immuable à la lecture. Le `record` ne protège pas de la mutabilité de ses composants — il protège seulement de la réaffectation. Un `record Commande(List<Article> articles)` dont la liste est un `ArrayList` exposé reste modifiable par l'appelant.

5.3 Le vrai mur : `OutOfMemoryError`

Un `OutOfMemoryError` n'est pas une exception : c'est une `Error`, et il ne se rattrape pas proprement. Le programme ci-dessous le provoque volontairement dans une JVM limitée à 256 Mo, pour montrer où se situe la limite réelle et pourquoi il faut la vérifier avant d'allouer.

PROGRAMME — LANCER AVEC `JAVA -XMX256M`

```
List<byte[]> fuite = new ArrayList<>();
int blocs = 0;
try {
    while (blocs < 100_000) {
        fuite.add(new byte[10 * 1024 * 1024]);   // 10 Mo
        blocs++;
    }
} catch (OutOfMemoryError e) {
    System.out.println("OutOfMemoryError après " + blocs + " blocs : " + e.getMessage());
}
```

SORTIE RÉELLE

```
OutOfMemoryError après 23 blocs de 10 Mo : Java heap space
```

23 blocs de 10 Mo, soit 230 Mo, dans un tas de 256 Mo. La leçon n'est pas « 23 » — ce nombre change à chaque machine — mais : **la dernière allocation réussie est toujours proche du plafond, et le dépassement est brutal**. Un service qui accepte un fichier de taille arbitraire ne tombera pas « un peu lentement » : il tombera d'un coup, et les requêtes en cours avec lui.

Ne rattrapez pas `OutOfMemoryError`

Un `catch (OutOfMemoryError e)` qui affiche un message et continue laisse l'application dans un état inconnu : la JVM peut avoir échoué au milieu d'une structure interne. La seule stratégie correcte est **préventive** : plafonner la taille des entrées (5.5), plafonner le nombre d'éléments (5.1), et limiter le tas par configuration. Si vous voulez vraiment observer l'erreur, faites-le dans un test dédié, avec un tas volontairement minuscule.

5.4 Chaînes et JSON : longueur et profondeur

Deux attaques classiques exploitent des limites oubliées : la chaîne construite en boucle (mémoire) et le document imbriqué (pile d'appels). La seconde porte un nom — JSON depth bomb — et un document de 20 000 caractères suffit à faire tomber un analyseur récursif naïf.

PROGRAMME — LA CHAÎNE QUI DOUBLE À CHAQUE TOUR

```
StringBuilder sb = new StringBuilder("x");
try {
    for (int i = 0; i < 40; i++) {
        sb.append(sb);
        if (i % 5 == 0) System.out.println("itération " + i + " : " + sb.length() + "
caractères");
    }
} catch (OutOfMemoryError e) {
    System.out.println("OutOfMemoryError : " + e.getMessage());
}
```

SORTIE RÉELLE (TAS DE 256 MO)

```
  itération 0 : 2 caractères
  itération 5 : 64 caractères
  itération 10 : 2048 caractères
  itération 15 : 65536 caractères
  itération 20 : 2097152 caractères
  itération 25 : 67108864 caractères
  OutOfMemoryError : Java heap space
```

Le doublement cache la croissance : entre l'itération 20 et l'itération 28, on passe de 2 millions à 500 millions de caractères. C'est exactement le profil d'une entrée malveillante « petite » (une recette de décompression, un motif de remplacement, un document imbriqué). La parade est toujours la même : un plafond exprimé en unité de la ressource, vérifié avant la croissance.

PROGRAMME — REFUSER PAR PROFONDEUR, AVANT D'ANALYSER

```
static class ProfondeurMaxException extends RuntimeException {
    ProfondeurMaxException(int max) { super("profondeur maximale dépassée : " + max); }
}

Object parseurSimple(String s, int profondeur) {
    if (profondeur > 100) throw new ProfondeurMaxException(100);
    if (s.startsWith("[")) return parseurSimple(s.substring(1, s.length() - 1), profondeur + 1);
    return s;
}

String bomb = "[".repeat(10_000) + "]".repeat(10_000);
parseurSimple(bomb, 0);
```

SORTIE RÉELLE

```
refus : profondeur maximale dépassée : 100
```

Ressource	Unité de plafond	Valeur de départ raisonnable
Document JSON	Profondeur d'imbrication	32 (une API classique ne dépasse pas 10)
Document JSON / XML	Taille en octets	1 Mo à 10 Mo selon l'usage
Chaîne de texte (commentaire)	Nombre de caractères	200 à 10 000
Collection d'entrée	Nombre d'éléments	5 (métier) · 10 000 (import technique)
Ligne de fichier	Nombre d'octets par ligne	4 096

> **Où se branche le plafond dans une application réelle**
>
> Le plus tôt possible, et à deux niveaux. **Niveau serveur** : la taille maximale d'une requête (par exemple `server.max-http-request-header-size` et la taille maximale de corps côté framework) rejette avant même votre code, avec un statut 413. **Niveau domaine** : votre objet validé refuse une collection de plus de 5 éléments, avec un message affichable. Les deux sont nécessaires : le premier protège la machine, le second protège la règle métier.

5.5 Fichiers et flux : la taille avant le contenu

Lire un fichier « pour voir » est l'erreur la plus coûteuse : on charge 2 Go en mémoire pour découvrir ensuite que le service n'accepte que 5 Mo. La taille se vérifie avant l'ouverture, avec les métadonnées du système de fichiers (`java.nio.file.Files.size`), et de nouveau pendant la lecture pour les flux qui ne sont pas des fichiers (requêtes HTTP).

PROGRAMME — LA TAILLE, PUIS LE VOLUME TOTAL, JAMAIS L'UN SANS L'AUTRE

```
static final long TAILLE_FICHIER_MAX = 5L * 1024 * 1024;      // 5 Mo
static final long LOT_MAX_OCTETS = 64L * 1024 * 1024;        // 64 Mo pour un lot

static void verifierTaille(String nom, long octets) {
    if (octets < 0) throw new IllegalArgumentException("taille négative");
    if (octets > TAILLE_FICHIER_MAX) {
        throw new IllegalArgumentException(nom + " : " + octets + " octets, maximum " +
TAILLE_FICHIER_MAX);
    }
}

static long estimerMemoire(int lignes, int octetsParLigne) {
    long total = (long) lignes * octetsParLigne;
    if (octetsParLigne > 4096) throw new IllegalArgumentException("ligne trop longue : " +
octetsParLigne);
    if (total > LOT_MAX_OCTETS) throw new IllegalArgumentException("lot de " + total + " octets,
maximum " + LOT_MAX_OCTETS);
    return total;
}
```

SORTIE RÉELLE

```
  1024 octets -> accepté
  5242880 octets -> accepté
  5242881 octets -> refus : photo.jpg : 5242881 octets, maximum 5242880
1000000 octets (1 000 lignes x 1 000 octets)
refus : lot de 80000000 octets, maximum 67108864
```

Trois pièges de la vérification de taille

1. Le `long` n'est pas décoratif : `lignes * octetsParLigne` en `int` déborde à partir de 2 Go — le produit devient négatif et passe tous les contrôles. Castez un opérande en `long` (c'est fait ci-dessus : `(long) lignes * octetsParLigne`). **2. La taille du fichier n'est pas la taille en mémoire** : un fichier UTF-8 de 5 Mo peut occuper 10 Mo de `char[]`, et un CSV de 5 Mo découpé en objets peut occuper 50 Mo. **3. La taille d'un flux n'est pas connue** : pour une requête HTTP, il faut compter les octets lus et s'arrêter au plafond, plutôt que se fier à un en-tête `Content-Length` fourni par le client.

6 Quotas, rythme et temps de traitement

> Une limite de valeur (« 5 objets ») ne protège pas d'une limite de fréquence : un utilisateur qui envoie mille fois 5 objets respecte toutes les règles métier et fait tomber le service. Ce chapitre traite les limites dans le temps : quota par période, débit, budget de traitement, taille de file.

6.1 Quota par utilisateur, par période

Un quota est un compteur avec une fenêtre. La forme la plus simple — un compteur remis à zéro toutes les 24 heures — a un défaut connu : on peut consommer deux fois le quota en quelques secondes à cheval sur la remise à zéro. La forme correcte est une **fenêtre glissante** : on ne garde que les événements des N dernières heures.

PROGRAMME — 3 DEMANDES PAR 24 H, ET UN SECOND UTILISATEUR INDÉPENDANT

```
static class Quota {
    private final int max;
    private final Duration fenetre;
    private final Map<String, Deque<LocalTime>> historique = new HashMap<>();

    void consommer(String utilisateur, LocalTime maintenant) {
        Deque<LocalTime> dq = historique.computeIfAbsent(utilisateur, k -> new ArrayDeque<>());
        purger(dq, maintenant);
        if (dq.size() >= max) {
            throw new QuotaDepasseException("quota de " + max + " par " + fenetre.toHours() + " h
atteint pour " + utilisateur);
        }
        dq.addLast(maintenant);
    }

    private void purger(Deque<LocalTime> dq, LocalTime maintenant) {
        while (!dq.isEmpty() && Duration.between(dq.peekFirst(), maintenant).compareTo(fenetre) >=
0) {
            dq.removeFirst();
        }
    }
}
```

SORTIE RÉELLE

```
  tentative 1 -> acceptée, reste 2
  tentative 2 -> acceptée, reste 1
  tentative 3 -> acceptée, reste 0
  tentative 4 -> refus : quota de 3 par 24 h atteint pour alice
  bob (autre utilisateur) -> 3 disponibles
```

6.2 Fenêtre glissante et seau à jetons

La fenêtre glissante garantit « au plus N par période », mais autorise un pic : si le quota est de 2 par heure, deux demandes peuvent partir à la même seconde. Quand il faut aussi lisser le débit, on passe au **seau à jetons** (token bucket) : un réservoir de capacité `C` se remplit de `débit` jetons par seconde ; chaque opération consomme un jeton.

PROGRAMME — 2 JETONS DE CAPACITÉ, 2 PAR SECONDE

```
static class Limiteur {
    private final double debitParSeconde;
    private final double capacite;
```

```
    private double jetons;
    private double dernier;

    boolean autoriser(double maintenant) {
        jetons = Math.min(capacite, jetons + (maintenant - dernier) * debitParSeconde);
        dernier = maintenant;
        if (jetons >= 1.0) { jetons -= 1.0; return true; }
        return false;
    }
}
```

SORTIE RÉELLE

```
  t=0.0s -> accepté (jetons restants 1.00)
  t=0.1s -> accepté (jetons restants 0.20)
  t=0.2s -> REFUSÉ (jetons restants 0.40)
  t=0.6s -> accepté (jetons restants 0.20)
  t=1.1s -> accepté (jetons restants 0.20)
  t=1.2s -> REFUSÉ (jetons restants 0.40)
```

Algorithme	Garantie	Usage typique
Compteur par fenêtre fixe	N par heure, avec un pic possible au changement de fenêtre	Quota de facturation, limite journalière simple
Fenêtre glissante	Jamais plus de N sur une période glissante	Quota métier (5 demandes par 24 h), anti-spam
Seau à jetons	Débit moyen lissé, avec une rafale bornée par la capacité	API publique, protection contre les rafales

Un quota par instance n'est pas un quota

Les deux implémentations ci-dessus gardent leur état en mémoire. Derrière un répartiteur de charge et trois instances, le quota réel est de 3 × 5. En production, l'état doit être partagé (base, cache distribué) ou le quota appliqué en amont, à l'entrée du système. De même, `HashMap` et `ArrayDeque` ne sont pas thread-safe : dans un service multi-thread, il faut une synchronisation ou une structure concurrente. C'est un exemple parfait de limite « correcte dans le code, fausse dans l'architecture ».

6.3 Budgéter le temps de traitement

Une entrée volumineuse ne provoque pas seulement de la mémoire : elle prend du temps. Un traitement qui doit répondre en 200 ms doit vérifier son budget en cours de route, sinon il finira par un dépassement de délai côté client — sans que votre code sache qu'il a échoué.

PROGRAMME — UN BUDGET DE 200 MS, VÉRIFIÉ ENTRE LES ÉTAPES

```
static class BudgetTemps {
    private final Duration max;
    private final long debut = System.nanoTime();

    Duration ecoule() { return Duration.ofNanos(System.nanoTime() - debut); }

    void verifier() {
        if (ecoule().compareTo(max) > 0) {
            throw new BudgetDepasseException("budget de " + max.toMillis() + " ms dépassé");
        }
    }
}
```

SORTIE RÉELLE

```
étape 1 ok (60 ms)
étape 2 ok (125 ms)
étape 3 ok (185 ms)
budget de 200 ms dépassé après 248 ms
durée réelle mesurée : 250 ms
```

Deux choix à assumer explicitement : **que fait-on quand le budget est dépassé ?** Soit on échoue (et le client reçoit une erreur), soit on tronque le traitement et on renvoie un résultat partiel marqué comme tel. **Où vérifie-t-on ?** Entre les étapes, aux points où le travail restant est borné — jamais au milieu d'une écriture en base.

> **Mesurer avec `System.nanoTime()`, pas `currentTimeMillis()`**
>
> `System.nanoTime()` est monotone : il ne recule pas quand l'horloge système est ajustée (synchronisation NTP, changement d'heure). `System.currentTimeMillis()` peut reculer, ce qui produit des durées négatives et des budgets qui ne se déclenchent jamais. Pour toute mesure de durée interne : `nanoTime`. Pour une date affichée ou stockée : `Clock` / `Instant`.

6.4 Files bornées : refuser plutôt que s'écrouler

Une file non bornée est une fuite de mémoire qui attend son heure : si les producteurs sont plus rapides que les consommateurs, elle grandit jusqu'à l'`OutOfMemoryError`. Une file bornée transforme la surcharge en refus explicite — ce qui est toujours préférable, parce que le refus est mesurable et peut être renvoyé au client.

PROGRAMME — CAPACITÉ 3, REFUS EXPLICITE, REPRISE APRÈS UN RETRAIT

```
static class FileBornee<T> {
    private final int capacite;
    private final Deque<T> elements = new ArrayDeque<>();

    void ajouter(T e) {
        if (elements.size() >= capacite) {
            throw new FilePleineException("file pleine (" + capacite + " éléments) : " + e + "
refusé");
        }
        elements.addLast(e);
    }
}
```

SORTIE RÉELLE

```
ajout a -> taille 1
ajout b -> taille 2
ajout c -> taille 3
ajout d -> refus : file pleine (3 éléments) : d refusé
retrait : a, taille 2
ajout d après un retrait -> taille 3
```

Stratégie de saturation	Comportement	Quand la choisir
Refuser (ce chapitre)	Exception immédiate, rien n'est perdu silencieusement	Traitement de demandes utilisateur : le client doit savoir que ce n'est pas passé
Bloquer (file bornée bloquante)	Le producteur attend, avec un délai maximal	Pipeline interne où la latence n'est pas critique
Écraser le plus ancien	Le nouvel élément remplace le plus vieux	Journal, cache, file d'événements où seul le récent compte
Non bornée	Grandit jusqu'à l'erreur fatale	Jamais, en production.

7 Gérer les erreurs : lever, attraper, envelopper

> Savoir qu'une valeur est invalide ne sert à rien si l'erreur est mal signalée, mal attrapée ou perdue en route. Ce chapitre est le cœur du document : choisir la bonne exception, la lever au bon endroit, l'envelopper sans perdre la cause, et ne jamais avaler une erreur.

7.1 Quand lever, quand retourner une valeur

La question n'est pas « exception ou pas », mais « l'appelant peut-il ignorer l'échec sans conséquence ? ».

Situation	Représentation	Pourquoi
Valeur absente, échec normal et fréquent	`Optional`, `OptionalInt`	Un champ facultatif, une recherche qui ne trouve rien. L'appelant doit décider, mais le programme n'est pas cassé.
Entrée invalide détectée à la frontière	Exception de validation (accumulée)	Il faut signaler l'erreur à l'utilisateur, avec les champs fautifs.
Argument illégal dans le programme	`IllegalArgumentException`, `Objects.requireNonNull`	C'est un bug d'appel : il doit apparaître en test, pas être stylisé.
État incompatible avec l'opération	`IllegalStateException` ou exception métier	« Déjà validée », « déjà fermée », « stock insuffisant » : le contexte est faux, pas l'argument.
Panne technique	Exception enveloppée + cause	Rien à corriger côté utilisateur ; il faut journaliser et échouer proprement.

Trois antimotifs à reconnaître

1. `catch (Exception e) { return null; }` — l'appelant ne peut plus rien comprendre, et le `NullPointerException` apparaît trois couches plus loin. **2.** Une exception attrapée puis remplacée par un `return false` — l'information « pourquoi » est perdue. **3.** Une méthode qui renvoie `-1` pour signaler une erreur : la valeur est à la fois un résultat et un signal, et personne ne la vérifie.

7.2 Choisir parmi les exceptions du JDK

Avant d'écrire une classe d'exception, vérifiez que le JDK n'en fournit pas déjà une. Les plus utiles dans le code de validation sont peu nombreuses :

Exception	Signification officielle (Javadoc)	Usage typique
`IllegalArgumentException`	Un argument est inapproprié	Valeur hors bornes, argument `null`, format invalide
`NullPointerException`	Une référence `null` là où un objet est attendu	Lever explicitement avec `Objects.requireNonNull(x, "message")`
`IllegalStateException`	L'état de l'objet ne permet pas l'appel	« Déjà payée », « déjà terminée », quota atteint dans l'objet
`UnsupportedOperationException`	L'opération n'est pas supportée	Collection immuable, méthode abstraite non implémentée
`IndexOutOfBoundsException`	Index hors des bornes d'un tableau ou d'une liste	Levé par le JDK ; à ne pas lever vous-même sans raison
`NumberFormatException`	Sous-classe d'`IllegalArgumentException` : la chaîne n'est pas un nombre	`parseInt`, `BigDecimal(String)`. **Jamais** laisser sortir ce message à l'utilisateur.
`DateTimeException`	Erreur de calcul ou de conversion date/heure	`plusDays` hors bornes, `LocalDate.of(2026,2,30)`
`ArithmeticException`	Condition arithmétique impossible	Division par zéro, `*Exact` qui déborde

PROGRAMME — LES EXCEPTIONS DU JDK EN ACTION, AVEC LEURS MESSAGES RÉELS

```
Objects.requireNonNull(null, "la référence est obligatoire");
List.of(1, 2, 3).get(5);
int[] t = new int[2]; t[3] = 1;
Integer.parseInt("12a");
throw new IllegalStateException("la commande est déjà validée");

System.out.println(Objects.requireNonNullElse(null, "défaut"));
```

SORTIE RÉELLE

```
java.lang.NullPointerException : la référence est obligatoire
java.lang.IndexOutOfBoundsException : Index 5 out of bounds for length 3
java.lang.ArrayIndexOutOfBoundsException : Index 3 out of bounds for length 2
java.lang.NumberFormatException : For input string: "12a"
java.lang.IllegalStateException : la commande est déjà validée
requireNonNullElse(null, "défaut") = défaut
```

Notez `requireNonNullElse` : quand une absence est normale, fournir une valeur par défaut explicite est meilleur que de lever. Et `requireNonNull` est toujours supérieur à une vérification manuelle, parce que le message est explicite et que l'exception décrit précisément l'élément fautif dans la pile d'appels.

7.3 Une hiérarchie d'exceptions maison

Le JDK couvre le générique ; votre domaine a besoin du spécifique. Une hiérarchie utile est **petite** : une base, un code, et deux ou trois spécialisations au plus.

FICHIERS — LA HIÉRARCHIE DU FIL ROUGE (RÉDUITE À L'ESSENTIEL)

```
public class ErreurMetier extends RuntimeException {
    private final CodeErreur code;
    public ErreurMetier(CodeErreur code, String message) { super(message); this.code = code; }
    public CodeErreur code() { return code; }
```

```
}

public class ErreurValidation extends RuntimeException {
    public record Violation(String champ, CodeErreur code, String message) {}
    private final List<Violation> violations;
    public ErreurValidation(List<Violation> violations) {
        super(violations.stream().map(Violation::toString).collect(Collectors.joining(" ; ")));
        this.violations = List.copyOf(violations);
    }
    public List<Violation> violations() { return violations; }
}

public class ChampInvalideException extends RuntimeException {   // erreur de forme
    private final String champ;
    public ChampInvalideException(String champ, String message, Throwable cause) {
        super(message, cause);
        this.champ = champ;
    }
    public String champ() { return champ; }
}
```

Trois règles pour une bonne exception maison

1. Un champ qui porte l'information exploitable : un code de catalogue, un nom de champ, une liste de violations. Sans cela, l'appelant ne peut que réafficher le message. **2. Un message construit à la construction** : il doit contenir la valeur reçue et la limite, pas seulement « erreur de validation ». **3. Hériter de `RuntimeException`** pour tout ce qui est validation : vous n'allez pas signer chaque méthode de l'application, et la validation est un échec normal, pas une panne. Réservez les exceptions vérifiées aux opérations d'entrée-sortie où l'appelant doit décider (voir 7.4).

PROGRAMME — LA HIÉRARCHIE EN ACTION : CODE, VIOLATION, CAUSE

```
try { new DemandeLocation("LOC-1", AUJ, 6, 1, 2, null); }
catch (ErreurMetier e) { System.out.println("refus [" + e.code() + "] " + e.getMessage()); }

try { DemandeLocation.depuisFormulaire(AUJ, "loc 2026", "2026-11-12", "9", "0", "12",
"x".repeat(201)); }
catch (ErreurValidation e) { e.violations().forEach(v -> System.out.println("  - " + v)); }

try { DemandeLocation.depuisLigne("LOC-1;2026-11-12;cinq;1;2"); }
catch (ChampInvalideException e) {
    System.out.println("champ=" + e.champ() + " cause=" +
e.getCause().getClass().getSimpleName());
}
```

SORTIE RÉELLE

```
refus [VALEUR_HORS_BORNES] jours hors bornes : 6
  - reference [FORMAT_INVALIDE] 3 à 20 caractères parmi A-Z, 0-9 et tiret : loc 2026
  - jours [VALEUR_HORS_BORNES] valeur attendue entre 1 et 5, reçu 9
  - articles [VALEUR_HORS_BORNES] valeur attendue entre 1 et 5, reçu 0
  - passagers [VALEUR_HORS_BORNES] valeur attendue entre 1 et 9, reçu 12
  - commentaire [VALEUR_HORS_BORNES] 200 caractères maximum, reçu 201
champ=jours cause=NumberFormatException
```

7.4 Attraper correctement

Attraper une exception est une décision de conception, pas un réflexe de sécurité. Quatre règles suffisent.

#	Règle	Justification
1	Attraper le type **le plus précis** possible	`catch (NumberFormatException e)` plutôt que `catch (RuntimeException e)` : vous n'attrapez pas ce que vous ne vouliez pas attraper.
2	Ne jamais attraper `Exception`, `Throwable`, `Error`	Vous intercepteriez `OutOfMemoryError`, `StackOverflowError`, et les bugs de votre propre code.
3	Un bloc `try` = une opération	Un `try` de 40 lignes cache l'origine de l'erreur ; découpez.
4	Si vous attrapez, vous faites quelque chose : traduire, enrichir, journaliser + relancer	Un `catch` vide est une panne silencieuse : la pire des catégories.

PROGRAMME — L'AVALEMENT SILENCIEUX, MESURÉ

```
static int totalAvecAvalement(List<String> entrees) {
    int total = 0;
    for (String s : entrees) {
        try {
            total += Integer.parseInt(s);
        } catch (NumberFormatException e) {
            // avalé : on continue comme si de rien n'était
        }
    }
    return total;
}

static int totalQuiEchoue(List<String> entrees) {
    int total = 0;
    for (String s : entrees) {
        try {
            total += Integer.parseInt(s);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("entrée non numérique : \"" + s + "\"", e);
        }
    }
    return total;
}
```

SORTIE RÉELLE

```
total (saisie correcte)   = 60
total (saisie avec "x")   = 40
total (sans avalement)    = 60
  refus explicite : entrée non numérique : "x"
```

La deuxième ligne est le cœur du sujet : avec `10`, `x`, `30`, le total « correct » est 40 au lieu de 70 — ou de 60 attendus par l'opérateur. Aucune erreur, aucun journal, aucune alerte : l'application a produit un chiffre faux, et personne ne le saura avant la comptabilité. C'est exactement ainsi que se fabriquent les bugs de production coûteux.

> **Le `catch` dans une boucle : réfléchissez à deux fois**
>
> Attraper par élément pour continuer le traitement est légitime (importer 10 000 lignes, ignorer les mauvaises et les compter), à condition de produire un rapport : nombre d'ignorées, premières lignes fautives, code d'erreur. Attraper pour continuer silencieusement, non.

7.5 Envelopper sans perdre la cause

Envelopper, c'est traduire une exception technique en exception de votre domaine — par exemple `FileNotFoundException` → `ConfigurationException`. Cela n'a de sens que si la cause d'origine est conservée : sans elle, le diagnostic devient impossible.

PROGRAMME — L'ENVELOPPE CORRECTE, ET LE `FINALLY` QUI AVALE TOUT

```
static String chargerConfiguration(String chemin) {
    try {
        throw new FileNotFoundException(chemin + " (No such file or directory)");
    } catch (FileNotFoundException e) {
        throw new ConfigurationException("impossible de charger la configuration : " + chemin, e);
    }
}

static int lectureAvecFinallyFautif() {
    try {
        throw new IllegalStateException("échec de lecture");
    } finally {
        return 0;   // le return du finally remplace l'exception
    }
}
```

SORTIE RÉELLE

```
message = impossible de charger la configuration : conf/app.properties
cause   = java.io.FileNotFoundException: conf/app.properties (No such file or directory)
  resultat = 0
```

La seconde méthode est un piège classique : un `return` (ou une exception) dans un bloc `finally` remplace ce qui se passait dans le `try`. L'exception disparaît entièrement — c'est pourquoi les outils d'analyse statique signalent tout `return` dans un `finally` comme une erreur.

PROGRAMME — `TRY-WITH-RESOURCES` : LES EXCEPTIONS SUPPRIMÉES SONT CONSERVÉES

```
try (Ressource r = new Ressource("fichier")) {
    r.lire();          // lève "échec pendant la lecture"
} catch (RuntimeException e) {
    System.out.println("principale : " + e.getMessage());
    System.out.println("supprimées : " + Arrays.toString(e.getSuppressed()));
}
```

SORTIE RÉELLE

```
  principale : échec pendant la lecture de fichier
  supprimées : [java.lang.IllegalStateException: échec pendant la fermeture de fichier]
```

La JLS (§14.20.3.2) décrit exactement ce mécanisme : si la fermeture échoue alors qu'une exception est déjà en cours, la seconde n'écrase pas la première, elle est supprimée et attachée via `addSuppressed`. C'est la raison pour laquelle `try-with-resources` est obligatoire dès qu'une ressource doit être fermée : écrit à la main, un `close()` dans un `finally` perd l'information.

PROGRAMME — RETROUVER LA CAUSE RACINE D'UNE CHAÎNE D'EXCEPTIONS

```
static void niveau1() { niveau2(); }
static void niveau2() { niveau3(); }
static void niveau3() { throw new IllegalStateException("transition interdite", new
IOException("flux fermé")); }

// dans le catch :
```

```
Throwable racine = e;
while (racine.getCause() != null) racine = racine.getCause();
System.out.println("racine = " + racine.getClass().getSimpleName() + " : " + racine.getMessage());
```

SORTIE RÉELLE

```
profondeur = 1 ; racine = IOException : flux fermé
```

7.6 Le coût d'une exception dans une boucle de jeu

Les exceptions ne sont pas « gratuites » : leur coût vient de la capture de la pile d'appels. Ce coût est négligeable pour une saisie utilisateur (une par seconde), et significatif pour une boucle qui tourne des millions de fois (un test, un import, une simulation).

PROGRAMME — 200 000 EXCEPTIONS, AVEC ET SANS PILE

```
RuntimeException lege = new RuntimeException("léger") {
    @Override public synchronized Throwable fillInStackTrace() { return this; }
};

long t0 = System.nanoTime();
for (int i = 0; i < n; i++) { try { throw new IllegalStateException("x"); } catch
(IllegalStateException e) { } }
long avec = (System.nanoTime() - t0) / 1_000_000;

long t1 = System.nanoTime();
for (int i = 0; i < n; i++) { try { throw lege; } catch (RuntimeException e) { } }
long sans = (System.nanoTime() - t1) / 1_000_000;
```

SORTIE RÉELLE

```
pile d'une exception créée 5 appels plus bas : 6 frames
fillInStackTrace() neutralisé  : 0 frames
200000 exceptions avec pile : 102 ms ; sans pile : 13 ms
```

Contexte	Exception comme contrôle de flux ?	Recommandation
Saisie utilisateur (≤ 1000 / s)	Oui, sans hésiter	Clarté avant tout : c'est le JDK lui-même qui lève (`parseInt`, `BigDecimal`)
Analyse de fichier (10^6 lignes)	À éviter	Tester avant d'appeler (`matches`), compter les rejets, ne lever qu'à la fin
Boucle de jeu temps réel	Non	Renvoyer un booléen ou un résultat ; garder les exceptions pour les cas anormaux
API de validation	Oui, c'est le contrat	Mais accumuler, pour ne lever qu'une fois par requête

Le raccourci qui ne sert à rien

Neutraliser `fillInStackTrace()` sur une exception métier réduit bien le coût (102 → 13 ms sur 200 000), mais vous perdez la pile — donc le fichier et la ligne — au moment précis où vous en avez besoin. À ne faire que sur une exception dont vous êtes certain de ne jamais avoir besoin de la pile : un résultat de validation interne, jamais une exception qui remonte à un `catch` de haut niveau.

Et `assert` ?

`assert` (JLS §14.10) est **désactivé par défaut** : sans l'option `-ea`, l'expression n'est même pas évaluée. Ce n'est donc pas un outil de validation d'entrée — c'est un outil de vérification interne pendant le développement. Vérifié : avec `jours = 9`, `assert jours <= 5 : "jours > 5 : " + jours;` ne produit rien sans `-ea`, et lève une `AssertionError` avec `-ea`. Ne l'utilisez jamais pour valider une saisie utilisateur.

Atelier — 15 min objectif : provoquer et lire cinq exceptions réelles

☐ Provoquez une `ArithmeticException` par division entière, puis par `Math.multiplyExact`. Comparez les messages : « / by zero » et « integer overflow ».

☐ Provoquez une `NumberFormatException` avec `Integer.parseInt(" 42")`, puis avec `"99999999999"`. Le message est le même : lequel des deux est un problème de format, lequel est un dépassement de type ?

☐ Écrivez une méthode qui enveloppe une `FileNotFoundException` dans votre propre exception, puis affichez la chaîne complète avec `getCause()` puis la cause racine.

☐ Rejouez la mesure de la section 7.6 : combien de temps pour 200 000 exceptions avec pile, sans pile, et pour un simple test `matches("\\d+")` qui évite l'exception ?

Ce que vous devez savoir lire : la première ligne d'une pile donne le type et le message ; la deuxième, le fichier et la ligne où l'exception a été levée — pas où elle a été attrapée. C'est ce couple que vous cherchez dans les journaux.

8 Étude de cas : un jeu (saisie, stock, triche)

> Un jeu est le meilleur terrain d'essai pour les limites d'entrée : l'utilisateur tape n'importe quoi, les quantités sont bornées par des règles inventées, le rythme compte, et le client n'est jamais digne de confiance. Quatre situations, quatre mécanismes — tous déjà vus, assemblés ici.

8.1 La boucle de saisie bornée

Le bug le plus fréquent d'une boucle de menu est la boucle infinie : l'utilisateur tape une lettre, `Scanner.nextInt()` lève `InputMismatchException`, le token fautif n'est pas consommé, et la boucle relit indéfiniment la même entrée. Le remède est triple : lire en `String`, convertir soi-même, et **borner le nombre d'essais**.

PROGRAMME — TROIS ESSAIS VALIDES, UN COMPTEUR, UNE SORTIE PROPRE

```
static int lireChoix(List<String> saisies, int min, int max, int maxEssais) {
    essais = 0;
    for (String s : saisies) {
        essais++;
        if (essais > maxEssais) break;
        try {
            int v = Integer.parseInt(s.strip());
            if (v < min || v > max) {
                System.out.println("  \"" + s + "\" hors bornes [" + min + "," + max + "]");
                continue;
            }
            return v;
        } catch (NumberFormatException e) {
            System.out.println("  \"" + s + "\" n'est pas un entier");
        }
    }
    throw new SaisieAbandonneeException("abandon après " + maxEssais + " essais invalides");
}
```

SORTIE RÉELLE

```
  "bonjour" n'est pas un entier
  "-3" hors bornes [1,5]
  "99" hors bornes [1,5]
  choix retenu : 3 après 4 tentatives
  abandon après 3 essais invalides
```

Règle de la boucle	Pourquoi
Lire en `String`, jamais `nextInt()`	`nextInt()` ne consomme pas le token invalide : la boucle le relit. En lisant la ligne puis en convertissant, chaque essai progresse.
Borner le nombre d'essais	Sans borne, un flux détaché ou un script automatique fait tourner la boucle à l'infini — et remplit les journaux.
Distinguer « pas un nombre » et « hors bornes »	Les deux messages n'ont pas la même utilité pour l'utilisateur ; et `NumberFormatException` recouvre aussi le dépassement de type.
Sortir par une exception, pas par `-1`	Une méthode qui renvoie `-1` pour signaler l'abandon oblige chaque appelant à s'en souvenir.

Et la fin de flux ?

`Scanner.hasNextLine()` renvoie `false` quand l'entrée est fermée (`Ctrl-D` dans un terminal, fin d'un fichier redirigé). C'est un cas à traiter explicitement, sinon un jeu lancé avec une entrée vide affiche « choix invalide » à l'infini. Testez toujours votre boucle de saisie avec un fichier vide : `java -cp out fr.exemple.Jeu < /dev/null`.

8.2 Stock, quantités et commandes par tour

Un jeu à objets combine trois limites qui se multiplient : le stock disponible, la quantité demandée, et le quota d'actions par tour. Chacune produit une exception différente, et l'ordre des vérifications compte.

PROGRAMME — LE STOCK D'ABORD, L'ÉTAT ENSUITE

```
void retirer(String objet, int n) {
    if (n <= 0) throw new IllegalArgumentException("quantité invalide : " + n);
    if (n > potions) throw new StockInsuffisantException(objet + " : " + n + " demandés, " +
potions + " disponibles");
    potions -= n;
}

void commander(String ordre) {
    if (commandes >= maxParTour) throw new QuotaTourDepasseException("3 commandes par tour
maximum : " + ordre + " ignorée");
    commandes++;
}
```

SORTIE RÉELLE

```
  après retrait de 2 : potions=1
  refus : potion : 2 demandés, 1 disponibles
  état inchangé : potions=1
  commande 1 acceptée (reste 2 ce tour)
  commande 2 acceptée (reste 1 ce tour)
  commande 3 acceptée (reste 0 ce tour)
  commande 4 refusée : 3 commandes par tour maximum : ordre 4 ignorée
  nouveau tour : reste 3 commandes
```

L'ordre des contrôles : argument, puis état, puis effet

`IllegalArgumentException` d'abord (l'appel est absurde : `n <= 0`), puis l'état du monde (`stock`, `quota`), puis seulement la modification. L'inverse — décrémenter puis vérifier — est le bug classique du stock négatif : `potions -= n` avant le contrôle laisse l'objet dans un état corrompu si l'exception part. La sortie ci-dessus le prouve : après le refus, l'état est inchangé.

PROGRAMME — LE QUOTA GLOBAL DE PARTIE, EN PLUS DU QUOTA PAR TOUR

```
void commander(String ordre) {
    if (commandes >= maxParTour) throw new QuotaTourDepasseException("3 commandes par tour
maximum : " + ordre + " ignorée");
    if (tour * maxParTour + commandes >= maxTotal) throw new QuotaTourDepasseException("partie
terminée : " + maxTotal + " commandes maximum");
    commandes++;
}
```

Deux limites, deux messages distincts : « vous avez épuisé vos actions de ce tour » et « la partie est terminée » ne se corrigent pas de la même façon. Un message unique pour deux causes différentes oblige le joueur à deviner.

8.3 Ne jamais croire le client

Dès qu'un jeu a un serveur, le client (navigateur, application mobile, ou simple `curl`) est un environnement hostile : n'importe qui peut envoyer un score de 999 999 points. Le serveur doit rejouer la logique de plausibilité — ce n'est pas de la méfiance, c'est le seul endroit où la règle est réellement applicable.

PROGRAMME — TROIS CODES D'ERREUR DISTINCTS CÔTÉ SERVEUR

```
int enregistrerScore(int points, int secondes) {
    if (points < 0 || secondes <= 0)
        throw new ScoreSuspectException("CHAMP_INVALIDE", "valeurs négatives ou nulles");
    if (points > SCORE_MAX_APPELE)
        throw new ScoreSuspectException("SCORE_HORS_BORNES", "score maximal de la partie : " +
SCORE_MAX_APPELE);
    int vitesse = points / Math.max(1, secondes);
    if (vitesse > VITESSE_MAX)
        throw new ScoreSuspectException("RYTHME_IMPOSSIBLE", "vitesse de " + vitesse + " points/s,
plausible maximum " + VITESSE_MAX);
    if (points <= meilleur)
        throw new ScoreSuspectException("REGRESSION", "score inférieur au record " + meilleur);
    meilleur = points;
    return meilleur;
}
```

SORTIE RÉELLE

```
client envoie : "j'ai gagné 999999 points, stock 42"
serveur : score maximal de la partie : 5000 (code SCORE_HORS_BORNES)
score accepté : 1400
serveur : score inférieur au record 1400 (code REGRESSION)
```

Contrôle côté serveur	Ce qu'il attrape	Ce qu'il n'attrape pas
Bornes de valeur	Score négatif, score impossiblement grand	Un score élevé mais plausible
Ratio points / durée	Un score obtenu « en 0 seconde »	Un bot qui joue lentement
Cohérence avec l'état	Un score qui régresse, un objet qui n'existe pas	Une duplication de requête (il faut un identifiant d'idempotence)
Rejeu serveur	La triche par modification du client, si les actions sont rejouables	La collecte automatisée si la politique le tolère

> **Le mot de la fin sur la confiance**
>
> Toute valeur qui traverse le réseau est une entrée utilisateur, même si elle vient de votre propre client. Les limites de l'interface (un curseur borné, un bouton grisé) sont de l'ergonomie, pas de la sécurité. En jeu comme en banque, la règle est identique : le contrôle qui compte est celui que l'attaquant ne peut pas contourner.

8.4 Hasard reproductible ou imprévisible

Dernière entrée souvent oubliée : la graine du générateur aléatoire. Ce n'est pas une entrée « utilisateur », mais c'est une entrée dont dépend l'intégralité de la partie — et la source d'une classe entière de vulnérabilités.

PROGRAMME — GRAINE FIXE, SUITE REJOUABLE ; `SECURERANDOM`, IMPRÉVISIBLE

```
RandomGenerator alea = RandomGeneratorFactory.of("L64X128MixRandom").create(42L);
for (int i = 0; i < 5; i++) System.out.print(alea.nextInt(6) + " ");

RandomGenerator alea2 = RandomGeneratorFactory.of("L64X128MixRandom").create(42L);   // rejouée
for (int i = 0; i < 5; i++) System.out.print(alea2.nextInt(6) + " ");

System.out.println(new SecureRandom().nextInt(6));
```

SORTIE RÉELLE

```
suite avec graine 42 : 0 1 0 0 3
rejouée à l'identique : 0 1 0 0 3
SecureRandom pour les gains : 1 (imprévisible, non rejouable)
```

Besoin	Générateur	Conséquence
Tests, simulations, rejeu d'une partie	`RandomGeneratorFactory…create(graine)` ou `new Random(graine)`	Suite identique : indispensable pour reproduire un bug signalé.
Jeu normal, sans enjeu financier	`ThreadLocalRandom.current()`, `RandomGeneratorFactory.of("L64X128MixRandom")`	Rapide, non rejouable. Ne pas l'utiliser pour un secret.
Gains, jetons, identifiants de session, loterie	`java.security.SecureRandom`	Imprévisible même si l'attaquant connaît l'algorithme. Plus lent, sans importance ici.

Pourquoi une graine « cachée » ne protège rien

Un générateur pseudo-aléatoire classique est entièrement déterminé par sa graine : quelques sorties observées suffisent à retrouver l'état interne, puis à prédire les tirages suivants. Cacher la graine dans le client ou dans l'horodatage ne change rien. Si un tirage a une valeur (un butin, un bonus, un tirage au sort), il doit être produit côté serveur avec `SecureRandom`, et le client ne reçoit que le résultat.

PROGRAMME — IDENTIFIANTS ET COMPTEURS : BORNÉS PAR CONSTRUCTION

```
for (int i = 0; i < 3; i++) System.out.println("id = " + UUID.randomUUID());

AtomicInteger compteur = new AtomicInteger();
System.out.println("essais : " + compteur.incrementAndGet());
```

SORTIE RÉELLE

```
id = bdca0fc4-74c8-4ab2-924b-b1412a5ac5ec
id = bf63e4df-2d66-437b-a6c9-e9ea34b44f3f
id = 1ff7a553-1908-49c6-8cf3-456f01dff2c7
compteur d'essais (atomique, sans exception) : 1
```

Notez `UUID.randomUUID()` pour les identifiants de partie : 122 bits aléatoires, pas de collision pratique, et l'identifiant ne révèle pas l'ordre de création (contrairement à un compteur séquentiel, qui permet de deviner le nombre de parties ou de deviner les identifiants voisins).

9 Lire une entrée : sources, lecteurs, plafonds

> Jusqu'ici, une entrée arrivait déjà sous forme de `String`. Mais d'où vient cette chaîne ? Entre le clavier et votre méthode de validation, il y a une source, un lecteur, un encodage et une fin de flux. C'est là que se cachent les bogues les plus difficiles à reproduire : la boucle infinie sur entrée vide, l'accent transformé en `?`, le BOM invisible qui rend le premier champ d'un CSV différent de tous les autres, le programme qui ne démarre pas parce qu'un paramètre obligatoire manque.

```
SOURCE          LECTEUR                          PRODUIT              CE QUI CASSE
──────────────────────────────────────────────────────────────────────────────────
String[] args   analyse manuelle                 valeurs typées       absent, --x=abc
console         BufferedReader + System.in       lignes de texte      Ctrl-D, écho
System.in       Scanner                          tokens, primitives   token non consommé
fichier         Files.readString / lines         texte, lignes        taille, BOM, CRLF
réseau          Reader derrière un plafond       texte borné          flux infini, octets
```

Ce chapitre ne traite aucune règle métier : il installe le tuyau qui amène la donnée jusqu'aux contrôleurs des chapitres 2 à 8. La règle qui le gouverne tient en une phrase : **toute lecture doit avoir une limite**, exprimée dans la signature de la méthode, et non dans un commentaire.

9.1 Le paysage : cinq sources, cinq lecteurs

Java propose quatre lecteurs de texte pour le prix d'un, et les choisir au hasard explique la plupart des surprises. Le tableau ci-dessous fixe les usages ; les sections suivantes les démontrent un par un.

Lecteur	Unité lue	Quand le choisir	Le piège principal
`String[] args`	Un argument complet	Options d'un exécutable, script, outil interne	Aucune vérification n'est faite pour vous : ni présence, ni format
`Scanner`	Un token selon un délimiteur	Saisie interactive simple, petits volumes, exercices	Un échec de conversion **ne consomme pas** le token fautif
`BufferedReader`	Une ligne (`readLine()`)	Gros volumes, protocoles ligne à ligne, parsing maîtrisé	Rend `null` en fin de flux, jamais une exception
`Files.readString`	Tout le fichier en mémoire	Petits fichiers de configuration, tests	Charge tout : à proscrire sans avoir vérifié `Files.size`
`Files.lines`	Un `Stream<String>` paresseux	Fichiers volumineux, traitement par lot	Le flux garde le fichier ouvert : `try-with-resources` obligatoire

> **Et `Console` ?**
>
> `System.console()` existe pour un besoin précis : lire un mot de passe sans écho, et écrire sur le terminal même quand `System.out` est redirigé. Il n'est **pas** utilisable partout : dans la plupart des environnements de test, des serveurs et des IDE, il vaut `null`. Nous le mesurons en 9.9, et nous ne l'utilisons jamais comme unique porte d'entrée.

9.2 `Scanner` : des tokens, un délimiteur, une position

`Scanner` découpe son entrée en tokens séparés par un délimiteur. Par défaut, ce délimiteur vaut `\p{javaWhitespace}+` : espaces, tabulations, retours à la ligne. Vous pouvez le vérifier sur une instance — puis constater que le premier appel à `nextLine()` juste après un `nextInt()` rend une chaîne vide.

PROGRAMME — TOKENS, DÉLIMITEUR PAR DÉFAUT, ET LE PIÈGE `NEXTINT`/`NEXTLINE`

```
Scanner sc = new Scanner("42 pomme 3.5");
System.out.println("delimiter : " + sc.delimiter());
System.out.println("hasNextInt() = " + sc.hasNextInt());
System.out.println("nextInt()    = " + sc.nextInt());
System.out.println("next()       = " + sc.next());
System.out.println("nextDouble() = " + sc.nextDouble());
System.out.println("hasNext()    = " + sc.hasNext());

// le piège : nextInt() s'arrête avant le \n, qui reste dans le tampon
Scanner p = new Scanner("42\nbonjour\n");
int n = p.nextInt();
String ligne = p.nextLine();
```

SORTIE RÉELLE

```
délimiteur par défaut : \p{javaWhitespace}+
hasNextInt()  = true
nextInt()     = 42
next()        = pomme
nextDouble()  = 3.5
hasNext()     = false
nextInt()  = 42
nextLine() = []  longueur 0
nextLine() = [bonjour]
```

La chaîne vide n'est pas un défaut de `Scanner` : `nextInt()` a lu `42` et s'est arrêté **avant** le `\n` délimiteur ; `nextLine()` consomme donc ce qu'il reste, c'est-à-dire rien. Ce décalage invisible est la cause n°1 des formulaires console qui « sautent » un champ. La règle : **liez toujours les entrées au même niveau** — tout en `nextLine()` et conversion à la main (ch. 8.1), ou tout en tokens, sans jamais mélanger.

Le délimiteur n'est pas décoratif : il définit ce qu'est un champ. Sur un fichier à point-virgule, laisser le délimiteur par défaut rend `5;5` comme un seul token. Et la locale décide de la signification du point décimal.

PROGRAMME — `USEDELIMITER` ET `USELOCALE`

```
Scanner a = new Scanner("LOC-11;2026-11-12;5;5");
a.useDelimiter(";");
List<String> champs = new ArrayList<>();
while (a.hasNext()) champs.add(a.next());

Scanner fr = new Scanner("12,35 1 234,00");
fr.useLocale(Locale.FRANCE);

Scanner us = new Scanner("12,35");
us.useLocale(Locale.US);
us.nextDouble();   // échoue
```

SORTIE RÉELLE (JVM DONT LA LOCALE PAR DÉFAUT EST `EN_US`)

```
champs = [LOC-11, 2026-11-12, 5, 5] (4)
locale FRANCE : "12,35" -> 12.35
locale FRANCE : "1 234,00" -> 1.0 puis 234.0  (l'espace sépare les tokens)
```

```
  locale US sur "12,35" : InputMismatchException
  défaut de la JVM  : en_US
```

Deux réglages à faire dès la construction

Pour une **interface** (quelqu'un tape à côté de vous) : `useLocale` avec la locale annoncée à l'utilisateur, et un délimiteur qui est la touche Entrée. Pour des **données** (fichier, réseau) : `useLocale(Locale.ROOT)` et un délimiteur explicite, sinon le même fichier produit des nombres différents selon la machine qui l'exécute. La troisième ligne de la sortie ci-dessus le montre : même avec la locale `FRANCE`, `1 234,00` reste deux tokens, car le séparateur de milliers n'est pas reconnu par `nextDouble` — le délimiteur l'emporte.

9.3 Les quatre pièges de `Scanner`

Aucun de ces pièges ne produit un message clair s'il n'est pas anticipé. Les quatre sont reproduits ci-dessous, avec la position du curseur après l'échec — c'est cette position, et non l'exception, qui décide si votre boucle progresse ou tourne en rond.

PROGRAMME — ÉCHEC DE CONVERSION, FLUX VIDE, MOTIF DE VALIDATION

```
Scanner sc = new Scanner("12,5 7");
try { sc.nextInt(); } catch (InputMismatchException e) { ... }
System.out.println("next() apres l'echec = [" + sc.next() + "]");

Scanner vide = new Scanner("");
vide.hasNext();     // false
vide.nextInt();     // NoSuchElementException

Pattern ref = Pattern.compile("LOC-\\d{4}-\\d{2}");
Scanner c = new Scanner("LOC-2026-11 XYZ-1 LOC-2027-03");
while (c.hasNext()) {
    if (c.hasNext(ref)) { ... c.next(ref); }
    else { String jete = c.next(); ... }
}
```

SORTIE RÉELLE

```
  InputMismatchException, position inchangée
  next() après l'échec = [12,5]  (relit le même token)
  hasNext() = false
  nextInt() = NoSuchElementException : null
  conforme : LOC-2026-11
  refusé   : XYZ-1 (consommé pour avancer)
  conforme : LOC-2027-03
  conformes = 2, refusés = 1
```

Piège	Symptôme	Remède
`InputMismatchException`	Le token fautif reste dans le tampon : la même erreur se répète à l'infini	Appeler `next()` pour consommer le token, ou lire des lignes et convertir soi-même
`NoSuchElementException`	Message `null`, pile inutile pour l'utilisateur	Tester `hasNext()` d'abord ; traiter la fin de flux comme un cas normal (9.9)
Mélange `nextInt`/`nextLine`	Un champ vide inattendu, un formulaire qui « saute » une ligne	Un seul niveau de lecture par flux
`next()` sans motif	`XYZ-1` accepté comme une référence valide	`hasNext(Pattern)` puis `next(Pattern)` : la forme est vérifiée à la lecture

Fermer un `Scanner`, c'est fermer `System.in`

`try (Scanner sc = new Scanner(System.in))` ferme le flux sous-jacent à la fin du bloc. Toute lecture ultérieure échoue alors avec `IOException : Stream closed` — y compris dans une autre classe qui croyait `System.in` disponible. Mesuré : après `sc.close()`, une nouvelle lecture rend exactement `IOException : Stream closed`. Ne fermez `System.in` que si vous possédez le processus.

9.4 `BufferedReader` et `Scanner` : lequel, et à quel prix

Les deux lisent du texte, mais pas au même coût. `Scanner` fait des expressions régulières sur chaque token ; `BufferedReader` découpe des lignes. Sur 200 000 lignes d'entiers, l'écart est d'un ordre de grandeur.

PROGRAMME — 200 000 LIGNES, TROIS FAÇONS DE LES ADDITIONNER

```
try (Scanner sc = new Scanner(bloc)) {
    while (sc.hasNextInt()) somme1 += sc.nextInt();
}
try (BufferedReader br = new BufferedReader(new StringReader(bloc), 1 << 16)) {
    String l;
    while ((l = br.readLine()) != null) somme2 += Integer.parseInt(l);
}
try (BufferedReader b2 = new BufferedReader(new StringReader(bloc))) {
    somme3 = b2.lines().mapToLong(Long::parseLong).sum();
}
```

SORTIE RÉELLE — JDK 25.0.5, UNE EXÉCUTION

```
  Scanner              176 ms  (somme 99900000)
  BufferedReader        13 ms  (somme 99900000)
  lines() + mapToLong   34 ms  (somme 99900000)
```

Les trois sommes sont égales : le résultat ne dépend pas du lecteur, seulement le temps. `Scanner` reste parfaitement adapté à une saisie interactive où l'utilisateur tape dix lignes ; il devient un goulot dès qu'un fichier dépasse quelques milliers de lignes, ou quand un service ingère des envois à haute fréquence. La règle pratique : **`Scanner` pour l'humain, `BufferedReader` pour la machine**.

Pourquoi `readLine()` rend `null` et non une chaîne vide

Une ligne vide existe : c'est un `\n` tout seul. Il fallait donc une valeur distincte pour « plus rien à lire ». `readLine()` rend `null` exactement une fois, en fin de flux ; un fichier qui se termine par un saut de ligne ne produit pas de dernière ligne vide. Sur l'entrée `"a\nb\n\nc\n"` : 4 lignes lues, dont 1 vide — et non 5.

9.5 Chaîne, octets, encodage : décoder est une décision

Un `String` n'existe pas dans un fichier : il y a des octets, et une règle pour les interpréter. Choisir la mauvaise règle ne lève pas toujours d'exception — c'est ce qui rend le défaut coûteux, car il voyage jusqu'à la base de données.

PROGRAMME — DOUZE OCTETS, TROIS LECTURES

```
byte[] octets = "Café crème".getBytes(StandardCharsets.UTF_8);
new String(octets, StandardCharsets.UTF_8);      // correct
new String(octets, StandardCharsets.ISO_8859_1); // mojibake, sans erreur
Charset.defaultCharset();                        // dépend de la JVM
```

SORTIE RÉELLE

```
  defaultCharset() = UTF-8
  UTF-8      -> Café crème (12 octets)
```

```
ISO-8859-1 -> CafÃ© crÃ¨me (12 caractères)
égales ? false
via InputStreamReader UTF-8 -> Café crème
```

Les douze octets sont les mêmes : la différence tient entièrement à la règle de décodage. Depuis Java 18, l'encodage par défaut de la JVM est UTF-8 (JEP 400), mais cela ne dispense pas de l'écrire : une bibliothèque, un conteneur ou un fichier produit ailleurs peut imposer autre chose. **Ne laissez jamais l'encodage implicite dans du code qui lit des données.**

PROGRAMME — LE REFUS EST UN SERVICE : `FILES.READSTRING` EST STRICT

```
Files.readString(latin1, StandardCharsets.UTF_8);   // fichier réellement en Latin-1
```

SORTIE RÉELLE

```
Files.readString(UTF-8) sur un fichier Latin-1 :
  MalformedInputException : Input length = 3
lu en ISO-8859-1 : LOC-2026-1é;5
decodeur REPLACE : LOC-2026-1�;5
```

Le même fichier donne trois résultats selon la politique choisie : exception (strict), texte correct (bon encodage), ou caractère de remplacement `�` (permissif). Les trois sont légitimes, mais un seul est juste à votre place. Pour des données métier — un montant, une référence, un nom — le mode strict est le bon défaut : il transforme une corruption silencieuse en une erreur que vous pouvez imputer à la bonne ligne.

Atelier — provoquer quatre défauts de lecture

Reproduire, sur votre machine, les quatre défauts qui coûtent le plus cher en production. Chacun tient en une ligne de commande.

1. **BOM invisible.** Créez un fichier avec `printf '\xEF\xBB\xBFref;jours\n'`. Lisez-le en UTF-8 : le premier caractère est `U+FEFF`, et le test `startsWith("ref")` est faux. Nettoyez avec `replace("\uFEFF", "")` en tête de fichier.
2. **Mauvais encodage.** Enregistrez un CSV en Latin-1, lisez-le en UTF-8 : `MalformedInputException`. Puis avec un décodeur `REPLACE` : la donnée est perdue, sans erreur.
3. **Fins de ligne CRLF.** Sur un fichier Windows, découpez avec `split("\n")` : le dernier champ vaut `"5\r"` et `Integer.parseInt` lève `NumberFormatException`.
4. **Fin de flux.** Lancez votre boucle de menu avec `< /dev/null` : `System.console()` vaut `null`, `available()` vaut `0`, et la première lecture rend `null`.

9.6 Lire un fichier : taille, lignes, BOM, fins de ligne

Un fichier se vérifie avant d'être lu. La taille, en octets, est disponible sans ouvrir le contenu : c'est la première limite à appliquer, et la plus économique.

PROGRAMME — `FILES.SIZE` AVANT `READSTRING`, ET LE BOM MESURÉ

```
for (Path p : List.of(utf8, bom, latin)) System.out.printf("  %-10s %3d octets%n",
p.getFileName(), Files.size(p));

String avecBom = Files.readString(bom, StandardCharsets.UTF_8);
String sansBom = Files.readString(utf8, StandardCharsets.UTF_8);
```

SORTIE RÉELLE

```
utf8.csv    24 octets
bom.csv     27 octets
latin1.csv  24 octets
```

```
  sans BOM : premier caractère = 'r' (U+0072)
  avec BOM : premier caractère = U+FEFF, longueur 25 vs 24
  premier champ lu sans nettoyage = [ref;jours]
  égal à "ref;jours" ? false
  après strip du BOM : true
```

Le fichier « avec BOM » pèse trois octets de plus — le BOM UTF-8 — et sa première ligne compte un caractère de plus. Un tableur, un éditeur Windows ou un export de fichier peuvent l'ajouter sans prévenir. Conséquence typique : le champ `ref` n'est pas trouvé dans l'en-tête, et tout le fichier est rejeté avec le message « colonne obligatoire manquante ». Le nettoyage tient dans une ligne, appliquée **une seule fois**, à l'ouverture.

PROGRAMME — `FILES.LINES` : GARDER TROIS LIGNES D'UN FICHIER DE 10 000

```java
int max = 3;
try (var flux = Files.lines(gros, StandardCharsets.UTF_8)) {
    List<String> gardees = flux.skip(1).limit(max).toList();
}
try (var flux = Files.lines(gros, StandardCharsets.UTF_8)) {
    System.out.println("lignes totales = " + flux.count());
}
```

SORTIE RÉELLE

```
  fichier de 158900 octets, on ne garde que 3 lignes :
    LOC-2026-0;1
    LOC-2026-1;1
    LOC-2026-2;1
  lignes totales comptées = 10001
```

> **`Files.lines` ne lit pas tout à l'appel — et garde le fichier ouvert**
>
> Le flux est **paresseux** : tant que rien ne consomme les lignes, rien n'est lu. C'est ce qui rend `limit(3)` efficace sur un fichier énorme. Mais tant que le `Stream` n'est pas fermé, le descripteur de fichier reste ouvert : sur un traitement de 10 000 fichiers, vous atteindrez `Too many open files`. D'où le `try-with-resources` systématique, y compris sur un flux.

9.7 `args` : la seule entrée qui vient sans mode d'emploi

Les arguments du programme arrivent sous forme de tableau de chaînes, déjà découpées par le shell. Aucune validation n'est fournie : ni présence, ni nombre, ni format. Le programme ci-dessous accepte `--ref=` et `--jours=`, et refuse tout le reste explicitement.

COMMANDE — QUATRE OPTIONS, DONT TROIS FAUTIVES

```
java -cp out fr.nice.lecture.Lectures --ref=LOC-2026-11 --jours=5 --jours=9 --jours=abc --verbose
```

SORTIE RÉELLE

```
== args reçus : 5 ==
  args[0] = [--ref=LOC-2026-11]
  args[1] = [--jours=5]
  args[2] = [--jours=9]
  args[3] = [--jours=abc]
  args[4] = [--verbose]
  --jours : "abc" n'est pas un entier
  argument inconnu, ignoré : --verbose
  ref = LOC-2026-11, jours = -1
  refus : jours hors bornes (1 à 5), reçu -1
```

Remarquez le comportement sur `--jours` répété : la **dernière** valeur l'emporte silencieusement. C'est acceptable pour un outil interne, pas pour une commande qui modifie des données : dans ce cas, une option répétée doit être une erreur, exactement comme un champ de formulaire fourni deux fois. Et notez que `jours = -1` traverse l'analyse avant d'être refusé : la borne métier (1 à 5, le chapitre 4) est appliquée après la conversion, jamais à sa place.

L'analyse tient en une boucle, mais chaque branche est une décision de contrat. Voici le code exact qui a produit la sortie ci-dessus — et la même analyse, réécrite pour **refuser** une option répétée au lieu de la laisser écraser la précédente.

PROGRAMME — ANALYSE MINIMALE, PUIS ANALYSE STRICTE

```
// 1. l'analyse qui produit la sortie ci-dessus
int jours = 5;                      // valeur par défaut, documentée
String ref = null;
for (String a : args) {
    if (a.startsWith("--jours=")) {
        String brut = a.substring("--jours=".length());
        try {
            jours = Integer.parseInt(brut);       // conversion
        } catch (NumberFormatException e) {
            jours = -1;                           // sentinelle, refusée plus bas
        }
    } else if (a.startsWith("--ref=")) {
        ref = a.substring("--ref=".length());
    } else {
        System.out.println("argument inconnu, ignoré : " + a);
    }
}
if (jours < 1 || jours > 5) { /* refus métier */ }

// 2. la même analyse, stricte : doublon refusé, option sans valeur distinguée
static int joursStrict(String[] args) {
    Integer jours = null;
    for (String a : args) {
        if (!a.startsWith("--jours")) continue;
        if (a.equals("--jours")) {
            throw new IllegalArgumentException("--jours attend une valeur (--jours=5)");
        }
        if (!a.startsWith("--jours=")) {
            throw new IllegalArgumentException("option inconnue : " + a);
        }
        if (jours != null) {
            throw new IllegalArgumentException("--jours fourni deux fois");
        }
        jours = Integer.parseInt(a.substring("--jours=".length()));
    }
    return jours == null ? 5 : jours;   // défaut seulement si absent
}
```

SORTIE RÉELLE — TROIS REFUS, DEUX ACCEPTATIONS

```
  --jours=3 --jours=4    -> --jours fourni deux fois
  --jours                -> --jours attend une valeur (--jours=5)
  --jours5               -> option inconnue : --jours5
  (aucun argument)       -> jours = 5
  --jours=3              -> jours = 3
```

> **`null` n'est pas « absent »**
>
> Dans la version stricte, `Integer jours = null` sert de témoin : `null` signifie « jamais fourni », et c'est seulement dans ce cas qu'on applique 5. Avec un `int jours = 5` initialisé, un `--jours=5` explicite serait indiscernable d'une absence — impossible de journaliser « valeur par défaut appliquée » ou de refuser un doublon. La même distinction vaut pour un champ de formulaire facultatif (ch. 4.3).

Contrôle sur `args`	Pourquoi il est indispensable
Nombre d'arguments	Aucun argument est le cas le plus fréquent : sans contrôle, `args[0]` lève `ArrayIndexOutOfBoundsException` avant tout message utile.
Option inconnue	Ignorer en silence transforme une faute de frappe (`--jour=5`) en valeur par défaut appliquée sans que personne ne le sache.
Forme `--clé=valeur`	Une option sans valeur doit être distinguée d'une valeur vide : `--ref=` est fourni mais invalide, `--ref` absent n'est pas la même erreur.
Conversion puis bornes	`NumberFormatException` pour « abc », refus métier pour 9 : deux messages, deux codes d'erreur, deux publics.

9.8 Borner la lecture : un lecteur qui refuse le trop-gros

Les plafonds du chapitre 5 s'appliquaient à des objets déjà en mémoire. Ici, la limite doit agir **pendant** la lecture : une seule ligne de 2 Go suffit à faire tomber la JVM avant que vous n'ayez pu compter quoi que ce soit. La classe ci-dessous place les plafonds dans la signature, et réutilise la hiérarchie d'exceptions du chapitre 7.

PROGRAMME — `LECTEURBORNE`, EXTRAIT (LE FICHIER COMPLET EST DANS `DEMO/SRC/FR/NICE/ENTREES/LECTURE/`)

```
public static LecteurBorne depuis(InputStream in, Charset charset) {
    var dec = charset.newDecoder()
            .onMalformedInput(CodingErrorAction.REPORT)
            .onUnmappableCharacter(CodingErrorAction.REPORT);
    return new LecteurBorne(new BufferedReader(new InputStreamReader(in, dec)));
}

public List<String> lire(int maxLignes, int maxOctetsParLigne) {
    List<String> out = new ArrayList<>();
    int cumulOctets = 0;
    String ligne;
    while ((ligne = br.readLine()) != null) {
        int octets = ligne.getBytes(StandardCharsets.UTF_8).length;
        if (octets > maxOctetsParLigne) {
            throw new FluxTropLongException("ligne trop longue : " + octets
                    + " octets, maximum " + maxOctetsParLigne, null);
        }
        if (out.size() == maxLignes) {
            throw new FluxTropLongException("plus de " + maxLignes + " lignes : maximum "
                    + maxLignes + " (" + cumulOctets + " octets déjà lus)", null);
        }
        cumulOctets += octets;
        out.add(ligne);
    }
    return List.copyOf(out);
}
```

SORTIE RÉELLE — SIX SCÉNARIOS

```
== 1. lecture nominale : 3 lignes lues, plafonds larges ==
  LOC-2026-11;2026-11-12;5;5
```

```
  LOC-2026-12;2026-11-13;2;1
  LOC-2026-13;2026-11-14;1;3

== 2. plafond de lignes : on s'arrête au 3e ==
  refusé : plus de 2 lignes : maximum 2 (52 octets déjà lus)

== 3. plafond de longueur de ligne ==
  refusé : ligne trop longue : 512 octets, maximum 200

== 4. octet invalide : refus, pas remplacement ==
  refusé : octet non décodable en UTF-8

== 5. la cause survit au passage de frontière ==
  message = octet non décodable en UTF-8
  cause   = java.nio.charset.MalformedInputException: Input length = 1

== 6. un flux vide n'est pas une erreur ==
  lignes = []
  lignes = [, , ]
```

Trois choix de conception méritent d'être justifiés, car ils se retrouvent dans n'importe quel lecteur de production :

Décision	Raison
Les plafonds sont des **paramètres**	Ils changent selon l'appelant (une ligne de journal, une ligne de CSV métier) ; les coder en dur oblige à dupliquer la classe.
L'erreur est levée **dès dépassement**, pas à la fin	Attendre la fin du flux sur une source infinie revient à ne jamais refuser. Le refus doit être aussi précoce que le plafond du chapitre 5.4.
Le décodage est **strict**	Un octet invalide est un défaut de la donnée : mieux vaut une exception imputable à la bonne ligne qu'un `?` inséré dans la base.
Un flux vide rend une liste vide	L'absence de données n'est pas une erreur ; la confondre avec un échec pousse à des fichiers « sentinelles » et à des `catch` inutiles.

> **Ce que le plafond de lignes ne protège pas**
>
> `readLine()` alloue la ligne entière **avant** de rendre la main : la mémoire consommée vaut la plus longue ligne du flux, pas le plafond. C'est pourquoi le contrôle de longueur doit rester un garde-fou applicatif, doublé d'une limite dure côté infrastructure (taille maximale de requête, `-Xmx`, quota du chapitre 6). Un octet mal intentionné ne se contente jamais d'une seule ligne de défense.

9.9 Console, `System.in` et fin de flux

L'entrée standard est un flux d'octets, éventuellement redirigé depuis un fichier ou un tuyau. Deux conséquences pratiques : il n'y a **pas** de console garantie, et la fin de flux est un cas normal, pas une panne.

PROGRAMME — LE MÊME CODE, DEUX EXÉCUTIONS

```
Console console = System.console();
System.out.println("System.console() = " + console);
System.out.println("octets disponibles = " + System.in.available());

try (BufferedReader br = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF_8))) {
    String l;
    while ((l = br.readLine()) != null && n < 5) { n++; ... }
}
```

SORTIE RÉELLE — ENTRÉE REDIRIGÉE DEPUIS UN FICHIER DE 5 LIGNES

```
== System.in en mode non interactif ==
  System.console() = null
  console == null ? true
  octets immédiatement disponibles = 29

== readLine() : null signifie fin de flux, pas ligne vide ==
  [1] "LOC-2026-11" longueur=11
  [2] "  42  " longueur=6
  [3] "" longueur=0
  [4] "Café" longueur=4
  [5] "fin" longueur=3
  boucle terminée après 5 ligne(s)

== fermer le lecteur ferme System.in : la suite est morte ==
  System.in fermé ? true
  nouvelle lecture -> IOException : Stream closed
```

COMMANDE — LA MÊME SANS AUCUNE ENTRÉE

```
java -cp out fr.nice.lecture.Lectures 8 < /dev/null
```

SORTIE RÉELLE

```
  System.console() = null
  console == null ? true
  octets immédiatement disponibles = 0
  boucle terminée après 0 ligne(s)
  read() sur flux vide = -1
```

La ligne `[3] "" longueur=0` est importante : une ligne vide est une donnée, pas une fin de flux. Une boucle qui traite `""` comme un arrêt s'arrête au premier paragraphe vide d'un fichier ; une boucle qui ignore `""` perd des enregistrements. Le seul signal de fin est `null` (ou `-1` pour `read()`).

Ne testez jamais une entrée interactive avec une entrée vide

C'est pourtant le test le plus utile : `< /dev/null`. Il révèle la boucle infinie qui affiche « choix invalide » sans fin, le `Scanner` qui relit le même token vide, et les messages d'erreur qui partent dans une boucle de journalisation. Ajoutez-le à vos tests : un programme qui ne sait pas s'arrêter proprement sur une entrée absente n'est pas prêt pour un environnement automatisé.

9.10 Décider vite : huit lignes pour choisir son lecteur

Situation	Outil	Premier réflexe
Un humain tape une valeur au clavier	`BufferedReader` + conversion manuelle	Lire une ligne, `strip()`, convertir, borner, 3 essais
Un menu interactif avec des nombres	`Scanner` si vous restez en tokens	Jamais de mélange avec `nextLine()`
Un fichier de 100 Mo à importer	`Files.lines` + plafonds	`Files.size` d'abord, `try-with-resources` ensuite
Un fichier de configuration de 2 Ko	`Files.readString`	Encodage explicite, BOM nettoyé en tête
Un mot de passe	`System.console().readPassword()`	Tester `null` et prévoir un repli non interactif
Des options de ligne de commande	Analyse maison sur `args`	Présence, inconnues, forme, puis conversion et bornes
Un flux réseau ou un tuyau	`BufferedReader` + `LecteurBorne`	Plafond de lignes et de longueur, décodage strict
Des données écrites par un autre système	Lecteur strict + validation du chapitre 4	Encodage et fins de ligne déclarés, jamais devinés

La règle à retenir

Une méthode qui lit une entrée doit répondre, dans sa signature, à trois questions : d'où (le flux), en quoi (l'encodage), et jusqu'où (les plafonds). Si l'une des trois manque, la limite sera appliquée ailleurs — ou pas du tout. C'est exactement le même raisonnement que pour la validation : ce qui n'est pas dans le type finit dans un commentaire, puis dans un incident.

10 Configuration : `.properties`, YAML, TOML

> Un fichier de configuration est la seule entrée qui décide du comportement de votre programme avant qu'il ne traite quoi que ce soit. Il a donc deux particularités : il arrive de l'extérieur (c'est une entrée comme une autre, chapitre 1), et il est écrit dans un format qui, s'il n'est pas typé, transformera votre `5` en booléen. Ce chapitre compare les trois formats que vous rencontrerez, avec les bibliothèques réellement utilisées en Java.

Le JDK ne lit que `.properties`

Il n'existe **aucune** API standard pour YAML ou TOML dans le JDK 25 : `java.util.Properties` est le seul lecteur intégré. Pour YAML, la bibliothèque de référence est **SnakeYAML 2.3** (`org.yaml:snakeyaml`) ; pour TOML, **tomlj 1.1.1** (`org.tomlj:tomlj`, qui dépend d'`org.antlr:antlr4-runtime`). Toutes les sorties de ce chapitre viennent de ces versions exactes, exécutées sur Temurin 25.0.5.

10.1 Ce que le format change : trois textes, trois résultats

La même intention — « cinq jours au maximum, un taux de 20 %, un service actif » — s'écrit différemment selon le format, et surtout se lit différemment. Le tableau ci-dessous résume ce que chaque format garantit ; les sections suivantes le prouvent par l'exécution.

Format	Typage	Tolérance aux fautes	Où il est utilisé
`.properties`	Aucun : tout est `String`	Très tolérant : une ligne douteuse est une clé	Java depuis toujours, Spring Boot, journaux, traductions
YAML	Implicite et surprenant : `05` est un entier, `NO` un booléen	Très tolérant : doublons et indentation approximative passent	Kubernetes, CI, Ansible, Spring Boot (`application.yml`)
TOML	Explicite et normalisé (dates, heures, entiers, flottants)	Strict : la moindre faute donne ligne et colonne	Cargo, `pyproject.toml`, outils modernes

10.2 YAML : les types que vous n'avez pas écrits

YAML n'a pas de schéma : c'est le texte qui décide du type, selon des règles héritées de YAML 1.1 puis révisées en 1.2. Un lecteur Java produit donc des objets dont vous n'avez pas choisi la classe. Voici le contenu réellement chargé par SnakeYAML avec le constructeur sûr (`SafeConstructor`).

PROGRAMME — UN FICHIER DE CONFIGURATION LISIBLE PAR UN HUMAIN… ET CE QU'EN FAIT LA MACHINE

```
service: locations
max_jours: 5
actif: true
taux: 0.20
vide:
  date: 2026-11-12
  remise: 1_000
  version: 1.10
devise: "CHF"
ref: LOC-2026-11
oui: yes
```

```
non: no
nul: null
pays: NO
```

SORTIE RÉELLE — TYPE JAVA PRODUIT POUR CHAQUE CLÉ

```
  service      String     locations
  max_jours    Integer    5
  articles_max Integer    5
  actif        Boolean    true
  taux         Double     0.2
  vide         LinkedHashMap {date=Thu Nov 12 00:00:00 UTC 2026, remise=1000, version=1.1}
  devise       String     CHF
  ref          String     LOC-2026-11
  oui          Boolean    true
  non          Boolean    false
  nul          null       null
  pays         Boolean    false
```

Quatre surprises dans ces douze lignes, toutes invisibles à la relecture du fichier : `2026-11-12` devient un `java.util.Date` (d'où le `Thu Nov 12`), `1_000` devient `1000`, `1.10` devient `1.1`, `yes`/`no` deviennent des booléens, et le code pays `NO` (Norvège) devient `false`. Ce dernier cas est un classique : la documentation de SnakeYAML s'appuie sur la spécification YAML 1.1, où `y|Y|yes|Yes|YES|n|N|no|No|NO|on|off` sont des booléens.

10.3 Les sept pièges de YAML, mesurés

PROGRAMME — SEPT FICHIERS, SEPT COMPORTEMENTS

```
code: 05                    // 1. un « code » à zéro initial
version: 1.10               // 2. une version, puis sa variante entre guillemets
a: yes  b: no  c: on  d: off  e: NO    // 3. booléens implicites
pays: NO                    // 4. un code pays
a:\n\tb: 1                  // 5. tabulation en indentation
jours: 5\njours: 9          // 6. clé dupliquée
base: &b {jours: 5}\nx: *b\ny: *b   // 7. ancre et alias
```

SORTIE RÉELLE

```
== 1. '05' perd son zéro : ce n'est pas une chaîne ==
  code = 5  (Integer)
== 2. la version 1.10 devient un nombre ==
  version = 1.1  (Double)
  entre guillemets = 1.10  (String)
== 3. yes/no/on/off : booléens en YAML 1.1, chaînes en 1.2 ==
  a -> true (Boolean)   b -> false (Boolean)   c -> true (Boolean)
  d -> false (Boolean)  e -> false (Boolean)
== 4. le code pays NO devient false ==
  pays       = false (Boolean)
  pays_quote = NO (String)
== 5. indentations : une tabulation suffit à casser le fichier ==
  tabulation -> ScannerException : while scanning for the next token
== 6. doublon de clé : accepté en silence, la dernière gagne ==
  jours = 9
== 7. ancre et alias : la même référence, pas une copie ==
  x == y (même objet) ? true
```

Piège YAML	Conséquence en production	Remède
`05`, `1.10`, `1_000`	Un code produit devient `5`, une version `1.1` : comparaisons et journaux faux	Guillemets explicites pour tout identifiant : `code: "05"`
`yes` / `NO`	Une devise ou un pays devient un booléen, ou l'inverse selon le lecteur	Guillemets + validation à la lecture (10.7)
Tabulation en indentation	Échec total du chargement, message technique (`ScannerException`)	Interdire les tabulations dans le fichier, et le vérifier en CI
Clé dupliquée	La dernière valeur gagne, sans avertissement : le déploiement applique autre chose que ce qu'on lit	Détecter les doublons avant chargement, ou refuser la clé en double
Ancre et alias	Deux clés partagent le même objet : modifier l'une modifie l'autre	Copier en profondeur à la lecture, ou éviter les alias dans les configs

10.4 TOML : typé, strict, et il vous dit où est la faute

TOML renverse la logique de YAML : les types sont **explicites dans la syntaxe** (une date est une date, une chaîne est entre guillemets), et une faute de syntaxe n'est jamais acceptée. L'API de tomlj rend des valeurs typées, avec `null` pour une clé absente — d'où la nécessité de distinguer `null` de zéro.

PROGRAMME — ACCÈS TYPÉ, ET LE PIÈGE DU `NULL`

```
TomlParseResult r = Toml.parse("""
        [service]
        nom = "locations"
        max_jours = 5
        actif = true
        taux = 0.20
        debut = 2026-11-12
        heure = 09:30:00
        version = "1.10"

        [service.limites]
        articles = 5
        """);

System.out.println(r.getLong("service.max_jours"));
System.out.println(r.getLocalDate("service.debut"));
System.out.println(r.getString("service.version"));
r.getLong("service.absent").intValue();   // clé absente
```

SORTIE RÉELLE

```
== TOML : erreurs de syntaxe ==
  hasErrors() = false
== accès typé ==
  long   service.max_jours  = 5 (Long)
  double service.taux       = 0.2
  bool   service.actif      = true
  String service.version    = 1.10
  date   service.debut      = 2026-11-12
  date   service.debut type = java.time.LocalDate
  heure  service.heure      = 09:30 (LocalTime)
  table  service.limites    = {articles=5}
== vérifier la présence AVANT de convertir ==
  service.absent      = null
  r.long("absent")     = null
```

```
  via int : 5
  getLong(absent).intValue() -> NullPointerException (la valeur est Long, pas long)
```

Deux enseignements. **Un** : `version = "1.10"` reste `1.10` — sans guillemets, TOML l'aurait considéré comme un flottant et la chaîne serait devenue `1.1`, exactement comme en YAML. Cette fois la faute se voit dans le fichier, parce que la syntaxe est explicite. **Deux** : `getLong` rend un `Long` (objet) et non un `long` : sur une clé absente, il rend `null` et l'appel `intValue()` lève `NullPointerException`. Il faut donc tester la présence — `r.contains("service.absent")` — ou utiliser `getLong(clé)` avec une valeur par défaut explicite.

PROGRAMME — SIX FICHIERS FAUTIFS, SIX DIAGNOSTICS AVEC LIGNE ET COLONNE

```
jours = 05
jours = 5\njours = 9
[service\njours = 5
jours = "5
jours =
date = 2026-02-30
```

SORTIE RÉELLE

```
  zéro initial : 05      TomlParseError ligne 1, col 10 : Unexpected '5', expected a newline or end-
of-input
  clé dupliquée          TomlParseError ligne 2, col 1 : jours previously defined at line 1, column 1
  table non fermée       TomlParseError ligne 1, col 9 : Unexpected end of line, expected ]
  guillemet non fermé    TomlParseError ligne 1, col 11 : Unexpected end of line, expected " or a
character
  valeur absente         TomlParseError ligne 1, col 9 : Unexpected end of line, expected ', ", ''',
""", a number, ...
  date impossible        TomlParseError ligne 1, col 16 : Invalid date 'FEBRUARY 30'
```

Ce que TOML refuse et qui vous protège

TOML 1.0 refuse `05` (un zéro initial n'est pas un entier valide), `1.` et `.5` (un flottant doit avoir des chiffres des deux côtés), la **clé dupliquée** — là où YAML l'accepte en silence — et les dates impossibles (`2026-02-30`), grâce au même `LocalDate` que le chapitre 3. Mesuré : `x = 1_000` vaut bien `1000` et `x = 0x1F` vaut `31` (les séparateurs et l'hexadécimal sont, eux, légitimes). En revanche, une tabulation en début de ligne est acceptée : TOML ne fait pas d'indentation significative.

10.5 `.properties` : le format du JDK, et ses quatre surprises

`java.util.Properties` est disponible partout, sans dépendance, et c'est sa seule qualité : tout y est une chaîne, y compris ce qui ressemble à un nombre. Deux comportements surprennent systématiquement.

PROGRAMME — ENCODAGE PAR DÉFAUT, ESPACES, GUILLEMETS, CLÉ ABSENTE

```
Path p = Files.createTempFile("conf", ".properties");
Files.writeString(p, "devise=CHF\nauteur=André\n", StandardCharsets.UTF_8);

new Properties().load(Files.newBufferedReader(p, StandardCharsets.UTF_8));
new Properties().load(Files.newInputStream(p));   // sans Reader
```

SORTIE RÉELLE

```
  via Reader UTF-8      : André
  via InputStream       : AndrÃ©
  (les octets sont UTF-8, la lecture est ISO-8859-1)

  [a] = [5]        [b] = [5]        [c] = [5]        [d] = [5]
  [e] = [sans egal]      [f] = ["5"]      [g] = [007]
```

```
  getProperty("absente") = null
  getProperty("absente", "defaut") = defaut
  Integer.parseInt(getProperty("absente")) -> NumberFormatException : Cannot parse null string
```

Surprise	Explication et règle
Encodage Latin-1 implicite	`load(InputStream)` décode en ISO-8859-1 depuis toujours (spécification de `Properties`) : « André » devient « AndrÃ© ». Chargez avec un `Reader` en UTF-8, ou passez à un autre format.
Espaces et séparateurs	`a = 5`, `b =5`, `c= 5` et même `d 5` (séparateur espace) donnent tous `5` ; deux-points est aussi un séparateur. Est-ce grave ? Non — mais un fichier peut « fonctionner » avec une syntaxe qu'on n'avait pas prévue.
Pas de guillemets	`f = "5"` vaut littéralement `"5"`, guillemets compris : le format n'a aucun échappement de ce genre. `g = 007` reste `007`.
Clé absente	`null`, jamais d'exception — donc `Integer.parseInt` lève `NumberFormatException : Cannot parse null string` bien plus loin. Utilisez `getProperty(clé, défaut)` puis validez.

10.6 Quatre sources, une seule valeur : la précédence

En pratique, une valeur vient de quatre endroits : les défauts du code, le fichier, les variables d'environnement (celles du conteneur), et les arguments de la ligne de commande (9.7). Sans règle explicite, vous ne savez pas ce qui s'applique — et le bug le plus coûteux est celui où le fichier dit 9, l'environnement dit 3, et le programme applique 5 parce qu'il a lu les défauts en dernier.

PROGRAMME — DÉFAUTS < FICHIER < ENVIRONNEMENT < ARGUMENTS

```
Map<String, String> defauts  = {max_jours=5, devise=EUR};
Map<String, String> fichier  = {max_jours=9, devise=CHF};
Map<String, String> env      = {MAX_JOURS=3};
String[] args                = {"--max_jours=4"};
```

SORTIE RÉELLE

```
  effectif : {max_jours=4, devise=CHF}
  max_jours  = 4    provenance = argument
  devise     = CHF  provenance = fichier

== et si le fichier gagnait : la borne s'applique quand même ==
  max_jours lu dans le fichier = 9
  refus au démarrage : max_jours hors bornes (1 à 5), reçu 9
  (un fichier de configuration est une ENTRÉE : il se valide comme les autres)
```

Deux règles, et elles suffisent

1. La précédence est fixée et documentée, du plus faible au plus fort : défauts du code → fichier → environnement → arguments. **2. La validation s'applique à la valeur effective**, une fois la fusion faite — pas couche par couche. C'est ce que montre la dernière ligne : peu importe que 9 vienne du fichier, la borne de 5 jours du chapitre 4 s'applique et le programme refuse de démarrer. Journaliser la provenance (comme ci-dessus) transforme un incident « pourquoi 4 jours ? » en une réponse immédiate.

10.7 Valider la configuration au démarrage

Une configuration se lit **une fois**, se valide **entièrement**, et produit un objet immuable. C'est exactement le schéma du chapitre 4, appliqué à une source de plus : accumuler les erreurs, puis

refuser de démarrer. Un service qui démarre avec un `max_jours` à `-1` ne tombera pas tout de suite ; il tombera à 3 h du matin, sur la première demande.

PROGRAMME — UN `RECORD` DE CONFIGURATION QUI NE PEUT PAS ÊTRE FAUX

```
record Config(String service, int maxJours, int articlesMax, BigDecimal taux, boolean actif) {
    static Config depuis(Map<String, String> sources) {
        List<String> erreurs = new ArrayList<>();
        String service = sources.getOrDefault("service", "");
        if (service.isBlank()) erreurs.add("service : obligatoire");
        int maxJours = entier(sources, "max_jours", 1, 5, erreurs);
        int articles = entier(sources, "articles_max", 1, 5, erreurs);
        BigDecimal taux = decimal(sources, "taux", new BigDecimal("0"), new BigDecimal("1"),
erreurs);
        boolean actif = booleen(sources, "actif", erreurs);
        if (!erreurs.isEmpty()) throw new ConfigurationInvalideException(erreurs);
        return new Config(service, maxJours, articles, taux, actif);
    }
}
```

SORTIE RÉELLE — UNE CONFIGURATION FAUTIVE, LES CINQ ERREURS D'UN COUP

```
  ConfigurationInvalideException (5 erreurs)
    service : obligatoire
    max_jours : entier attendu, reçu "neuf"
    articles_max : hors bornes (1 à 5), reçu 9
    taux : hors bornes (0 à 1), reçu 1.20
    actif : booléen attendu (true/false), reçu "peut-être"
  démarrage refusé — aucun client n'a été servi

== configuration complète : acceptée ==
  Config[service=locations, maxJours=5, articlesMax=3, taux=0.20, actif=false]

== valeurs par défaut appliquées quand tout est absent ==
  Config[service=locations, maxJours=5, articlesMax=5, taux=0, actif=true]
```

> **Le même réflexe que pour une saisie**
>
> Les valeurs absentes ont un **défaut explicite**, les valeurs présentes sont **converties puis bornées**, les erreurs sont **accumulées** avec le nom de la clé, et l'échec est **une exception** levée avant la première requête (chapitre 7 : enveloppée, avec sa cause, jamais avalée). Une configuration qui se charge sans validation n'est pas une configuration : c'est une hypothèse.

10.8 YAML hostile : la bombe et le tag

Un fichier de configuration peut venir d'ailleurs : un dépôt, un pull request, un tableau de bord d'administration, une variable d'environnement. YAML a deux fonctionnalités qui deviennent des armes dès que la source n'est pas de confiance : les alias, et les tags de type.

PROGRAMME — 511 CARACTÈRES QUI PEUVENT SATURER LA MÉMOIRE, ET UN TAG DE TYPE ARBITRAIRE

```
LoaderOptions limite = new LoaderOptions();
limite.setMaxAliasesForCollections(50);
new Yaml(new SafeConstructor(limite)).load(in);

LoaderOptions sansLimite = new LoaderOptions();
sansLimite.setMaxAliasesForCollections(1000);
new Yaml(new SafeConstructor(sansLimite)).load(in);
```

SORTIE RÉELLE

```
== bombe de références (billion laughs) ==
  fichier de 511 caractères
  avec maxAliasesForCollections(50) : refusé en 54 ms
    Number of aliases for non-scalar nodes exceeds the specified max=50
  avec la limite à 1000 : chargé en 5 ms

== tag global : SafeConstructor refuse, Constructor accepte ==
  safe : ComposerException : Global tag is not allowed:
tag:yaml.org,2002:javax.script.ScriptEngineManager
  défaut (new Yaml()) : ComposerException
```

Ne chargez jamais du YAML non fiable avec le constructeur par défaut

SnakeYAML a rendu la classe `Constructor` capable d'instancier des classes arbitraires : c'est la vulnérabilité **CVE-2022-1471** (sévérité élevée, exécution de code à distance), corrigée en **2.0** — toute version `>= 0 < 2.0` est affectée. Trois règles : utilisez `SafeConstructor`, gardez la limite d'alias (`setMaxAliasesForCollections`), et bornez la taille du fichier avant de le lire, comme en 9.6. SnakeYAML 2.x refuse d'ailleurs les tags globaux par défaut — c'est ce que montre la dernière ligne de la sortie.

10.9 Décider vite : quel format, et avec quel garde-fou

Situation	Format	Garde-fou minimum
Quelques clés plates, aucun type	`.properties`	`Reader` en UTF-8, jamais `InputStream` seul
Structure imbriquée lue par un humain	YAML	Guillemets sur tout identifiant, doublons refusés
Config d'outil, dates et heures	TOML	`hasErrors()` d'abord, puis présence avant conversion
Fichier fourni par un tiers	TOML, ou YAML avec `SafeConstructor`	Taille bornée, `maxAliasesForCollections`, tags refusés
Configuration d'entreprise (Spring, Quarkus…)	Celui du framework	La validation déclarative ne dispense pas des invariants (10.7)

Atelier — quatre vérifications à faire sur votre propre configuration

Trente minutes, sans écrire de code : il s'agit de regarder ce que votre application lit vraiment.

1. **Cherchez les identifiants non guillemets** dans vos YAML : codes produits, versions, codes pays. Un identifiant qui commence par `0` ou contient un point a probablement changé de type.
2. **Dupliquez une clé** volontairement, puis démarrez : si rien ne se plaint, votre configuration peut mentir. Ajoutez la détection.
3. **Videz une valeur** (`max_jours:`) et un type (`actif: peut-être`) : que se passe-t-il ? Un démarrage réussi est le mauvais résultat.
4. **Lancez avec une variable d'environnement contradictoire** et vérifiez que la précédence est celle que vous croyez — puis journalisez la provenance.

La règle à retenir. Un fichier de configuration n'est pas du code : c'est une **entrée**. Il se borne en taille, se lit avec un encodage explicite, se convertit avec un format qui distingue les types, se fusionne selon une précédence documentée, et se valide avant la première requête. Les formats ne diffèrent que par l'endroit où ils placent la faute : dans le type (YAML), dans la syntaxe (TOML), ou nulle part (`.properties`).

11 Cadrer l'output

> Une entrée bien validée peut encore produire une sortie inutilisable : un tableau qui se décale à la première valeur longue, un CSV cassé par une virgule dans un nom, un accent transformé en `?`, un message d'erreur technique affiché à un client. C'est le dernier maillon, et c'est celui que l'utilisateur voit.

11.1 Un format unique, fixé et testé

Un output destiné à un humain peut être joli ; un output destiné à un autre programme doit être **rigoureusement régulier**. Le tableau ci-dessous est un exemple de format à largeur fixe : une ligne d'en-tête, des séparateurs, et des colonnes de largeur constante.

PROGRAMME — UN TABLEAU À LARGEURS FIXES, AVEC EN-TÊTE

```
static void ligne(String ref, String debut, int jours, int articles, String montant) {
    System.out.printf("  | %-11s | %-10s | %4d | %4d | %8s |%n", ref, debut, jours, articles,
montant);
}
```

SORTIE RÉELLE

```
  +-------------+------------+------+------+----------+
  | référence   | début      | jrs  | art. | montant  |
  +-------------+------------+------+------+----------+
  | LOC-2026-11 | 2026-11-12 |    5 |    5 |    12.50 |
  | LOC-2026-12 | 2026-11-13 |    2 |    1 |     8.00 |
  | LOC-2026-13 | 2026-11-14 |    1 |    3 | 1 234.00 |
  +-------------+------------+------+------+----------+

    +-------------+------------+------+------+----------+
```

La largeur est un minimum, pas un maximum

Troisième ligne du tableau : « 1 234.00 » fait 8 caractères, mais avec un montant plus grand, la colonne se décale — `%-8s` ne tronque pas, il complète à gauche avec des espaces. Et pour une chaîne toujours plus grande que la colonne, le tableau se disloque. Si le format doit être stable, il faut soit tronquer explicitement (`tronquer(ref, 11)`), soit aligner sur la plus grande valeur calculée à l'avance. C'est exactement le même choix que dans un rapport imprimé.

11.2 `printf` : largeurs, précision, erreurs de format

`java.util.Formatter` est puissant mais non typé : une erreur de spécificateur n'est détectée qu'à l'exécution, et certaines sont silencieuses.

PROGRAMME — LES SPÉCIFICATEURS UTILES, PUIS LES TROIS ERREURS

```
System.out.printf("  [%5d] [%-5d] [%05d]%n", 42, 42, 42);
System.out.printf("  [%.2f] [%8.2f] [%,d]%n", 12.3456, 12.3456, 1234567);
System.out.printf("  [%s] [%10s] [%-10s]%n", "ab", "ab", "ab");
System.out.printf("  [%c] [%b] [%x] [%o]%n", 'A', true, 255, 8);

try { System.out.printf("%d%n", "pas un nombre"); }
catch (IllegalFormatConversionException e) { System.out.println("  %d avec une String -> " +
e.getMessage()); }

try { System.out.printf("%d%n"); }
```

```
catch (MissingFormatArgumentException e) { System.out.println("  argument manquant -> " +
e.getMessage()); }

System.out.println("  String.format(\"%s\", (Object) null) = \"" + String.format("%s", (Object)
null) + "\"");
```

SORTIE RÉELLE (LOCALE PAR DÉFAUT, EN_US)

```
[   42] [42   ] [00042]
[12.35] [   12.35] [1,234,567]
[ab] [        ab] [ab        ]
[A] [true] [ff] [10]
%d avec une String -> IllegalFormatConversionException : d != java.lang.String
argument manquant -> MissingFormatArgumentException : Format specifier '%d'
String.format("%s", (Object) null) = "null"
```

Avec la locale française, les deux premières lignes deviennent `[12,35] [ 12,35] [1 234 567]` : le séparateur décimal et le séparateur de milliers changent. Ce n'est pas un détail cosmétique — un fichier de test qui contient `12.35` échouera sur une machine configurée en français.

Trois règles pour un output stable

1. Toujours une locale explicite pour les nombres : `String.format(Locale.ROOT, "%.2f", x)` pour un fichier ou une API, `Locale.FRANCE` pour un affichage utilisateur. Jamais la locale par défaut. **2. Jamais `"" + objet` pour un `null` attendu** : le résultat est la chaîne « null », qui se propage dans les exports. Utilisez `Objects.toString(obj, "")`. **3. `%s` sur un objet appelle `toString()`** : si `toString()` n'est pas défini, l'utilisateur voit `fr.acme.Commande@1b6d3586`.

11.3 Les délimiteurs : `Scanner` et le CSV

Deux problèmes symétriques : lire un fichier délimité, et en écrire un. Dans les deux cas, le délimiteur peut apparaître dans les données.

PROGRAMME — `SCANNER` ET SES DÉLIMITEURS

```
Scanner s1 = new Scanner("a b\tc");
while (s1.hasNext()) System.out.print("\"" + s1.next() + "\" ");

Scanner s2 = new Scanner("Café;4.50;3").useDelimiter(";");
System.out.println(s2.next() + " / " + s2.next() + " / " + s2.next());

Scanner s3 = new Scanner("42,5").useDelimiter(Pattern.compile("[;\\n]"));
s3.nextInt();    // locale française
```

SORTIE RÉELLE

```
next() : "a" "b" "c"
délimiteur ";" : Café / 4.50 / 3
nextInt() sur "42,5" -> InputMismatchException (locale française, virgule)
```

La Javadoc de `Scanner` le dit : le délimiteur par défaut « matches whitespace », donc espace, tabulation, retour à la ligne — pas le point-virgule. Et les conversions numériques de `Scanner` suivent les conventions de la locale par défaut : en français, `nextDouble()` accepte `42,5` et refuse `42.5` ; en anglais, l'inverse (mesuré en 9.2). Pour `nextInt()`, aucun des deux n'est un entier. Pour un fichier de données, fixez la locale : `scanner.useLocale(Locale.ROOT)`.

PROGRAMME — ÉCRIRE DU CSV SANS LE CASSER

```
static String echapperCsv(String v) {
    if (v.contains(";") || v.contains("\"") || v.contains("\n")) {
        return "\"" + v.replace("\"", "\"\"") + "\"";
```

```
    }
    return v;
}

for (String v : new String[] { "Café", "Thé, vert", "Lait \"bio\"", "Ligne\ncassée" }) {
    System.out.println(v + ";" + v.length());              // naïf
    System.out.println(echapperCsv(v) + ";" + v.length());  // correct
}
```

SORTIE RÉELLE

```
    Café;4
    Thé, vert;9
    Lait "bio";10
    Ligne
cassée;12
  CSV correct :
    Café;4
    Thé, vert;9
    "Lait ""bio""";10
    "Ligne
cassée";12
```

Contrainte CSV	Règle (RFC 4180, reprise par tous les tableurs)
Le délimiteur dans une valeur	Entourer la valeur de guillemets doubles.
Un guillemet dans une valeur	Le doubler (`"` devient `""`), et entourer la valeur de guillemets.
Un saut de ligne dans une valeur	Entourer de guillemets : le champ peut alors contenir un saut de ligne, et le fichier reste valide.
Une valeur qui commence par `=`, `+`, `-`, `@`	Préfixer d'une apostrophe : sinon un tableur peut l'interpréter comme une formule (injection CSV).

Ne fabriquez pas votre CSV à la main pour un usage réel

Le code ci-dessus illustre le mécanisme, mais l'échappement CSV est plus subtil qu'il n'y paraît (BOM, fins de ligne, encodage, champs `null` à distinguer de `""`). En production, utilisez une bibliothèque dédiée (par exemple Apache Commons CSV, ou le support d'export de votre framework) et écrivez toujours l'en-tête. Même remarque pour JSON : aucune concaténation de chaînes, un sérialiseur, et une vérification d'échappement sur les valeurs.

11.4 Encodage et Unicode : l'output qui devient illisible

Un accent qui devient `?` dans un fichier ou une console est un problème d'encodage, et il est invisible tant qu'on travaille dans une seule configuration. Depuis Java 18, l'encodage par défaut de la JVM est UTF-8 (JEP 400), mais la console, elle, dépend de la machine.

PROGRAMME — LES PROPRIÉTÉS QUI GOUVERNENT L'AFFICHAGE, ET LE TEST RÉEL

```
System.out.println("file.encoding   = " + System.getProperty("file.encoding"));
System.out.println("stdout.encoding = " + System.getProperty("stdout.encoding"));
System.out.println("native.encoding = " + System.getProperty("native.encoding"));
System.out.println(Charset.defaultCharset());

String texte = "Café — 5 €";
System.out.println("écrit tel quel    : " + texte);
System.out.println("octets en UTF-8   : " +
```

```
Arrays.toString(texte.getBytes(StandardCharsets.UTF_8)));
System.out.println("chars : " + texte.length() + " ; points de code : " + texte.codePointCount(0,
texte.length()));
```

SORTIE RÉELLE — AVEC UN TERMINAL UTF-8

```
file.encoding   = UTF-8
stdout.encoding = UTF-8
native.encoding = UTF-8
UTF-8
écrit tel quel    : Café — 5 €
octets en UTF-8   : [67, 97, 102, -61, -87, 32, -30, -128, -108, 32, 53, 32, -30, -126, -84]
chars : 10 ; points de code : 10
```

LA MÊME COMMANDE AVEC `LC_ALL=C`

```
file.encoding   = UTF-8
stdout.encoding = ANSI_X3.4-1968
native.encoding = ANSI_X3.4-1968
?crit tel quel    : Caf? ? 5 ?
chars : 10 ; points de code : 10
```

Analyse : les **octets sont corrects** dans les deux cas — l'application a produit de l'UTF-8 valide, comme le montrent les 15 octets. C'est le terminal qui ne sait pas les afficher, et qui substitue `?`. La leçon : ne cherchez pas le bug dans votre code si `getBytes(UTF_8)` donne les bons octets ; vérifiez la configuration du terminal, du fichier ou du client.

Situation	Cause probable	Correctif
`?` sur la console	`stdout.encoding` ne sait pas représenter le caractère	Terminal en UTF-8, ou `-Dstdout.encoding=UTF-8`
`Ã©` au lieu de `é`	Double décodage : des octets UTF-8 relus en ISO-8859-1	Lire le fichier avec le bon `Charset` : `Files.readString(path, StandardCharsets.UTF_8)`
Un rectangle vide	Police sans le glyphe (emoji, caractère exotique)	Changer de police ; pour un PDF, vérifier que la police est embarquée
Un accent « différent » non trouvé	Forme Unicode décomposée (e + accent)	`Normalizer.normalize(s, Form.NFC)` avant comparaison
Un caractère hors BMP qui compte 2	UTF-16 : un point de code hors BMP = 2 `char`	`codePointCount` pour les limites visuelles

PROGRAMME — DEUX CHAÎNES IDENTIQUES À L'ŒIL, DIFFÉRENTES EN MÉMOIRE

```
System.out.println("\"e\\u0301\" vs \"é\" : " + "e\u0301".equals("é"));
System.out.println("normalisés identiques ? "
        + Normalizer.normalize("e\u0301", Normalizer.Form.NFC).equals("é"));
String emoji = "\uD83D\uDC4D";        // un caractère hors BMP, écrit en échappement
System.out.println("length() = " + emoji.length()
        + " ; codePointCount = " + emoji.codePointCount(0, emoji.length()));
```

SORTIE RÉELLE

```
"e\u0301" vs "é" : false
normalisés identiques ? true
length() = 2 ; codePointCount = 1
```

11.5 Message utilisateur, message technique

Une même erreur a besoin de deux textes, et les mélanger est l'erreur la plus fréquente des API : soit le client reçoit `NumberFormatException: For input string: "4 2"`, soit les journaux reçoivent « Oups, quelque chose s'est mal passé » et plus personne ne peut diagnostiquer.

PROGRAMME — UN OBJET D'ERREUR QUI PORTE LES DEUX VERSIONS

```
record ErreurFormatee(String champ, String messageUtilisateur, String code) {
    String pourUtilisateur() { return messageUtilisateur + " (champ : " + champ + ")"; }
    String pourLogs() { return "code=" + code + " champ=" + champ + " message=\"" +
messageUtilisateur + "\""; }
}
```

SORTIE RÉELLE

```
  pour l'utilisateur : La durée maximale est de 5 jours. Vous avez demandé 9 jours. (champ : jours)
  pour les logs      : code=VALEUR_HORS_BORNES champ=jours message="La durée maximale est de 5 jours.
Vous avez demandé 9 jours."
```

Élément	Message utilisateur	Message technique (journal)
Contenu	Ce qui est faux, la limite, la valeur reçue	Code stable, champ, valeur, exception d'origine
Langue	Celle de l'utilisateur	Une seule langue (anglais), pour la recherche dans les journaux
Stabilité	Peut changer à chaque version	**Doit** être stable : les alertes s'appuient sur le code, pas sur le texte
Identifiant de corrélation	Facultatif, mais utile au support	Systématique : identifiant de requête, d'utilisateur, d'objet
Données sensibles	Jamais (pas de valeur de carte, pas de mot de passe)	Jamais non plus — journaliser une valeur masquée

Le format à retenir pour une API

Un corps d'erreur structuré, avec un code stable, une liste de violations (`champ`, `code`, `message`), et un identifiant de corrélation. Le message est destiné à l'affichage ; le code est destiné aux traitements automatiques. Cette séparation est celle que décrit la norme des « Problem Details » pour HTTP (RFC 9457) — inutile de l'inventer, autant s'y conformer.

Atelier — 25 min objectif : un output d'erreur stable et testable

☐ Reprenez l'`ErreurValidation` du chapitre 7 et produisez une sortie texte, une ligne par violation, au format `champ | code | message`.

☐ Écrivez un test JUnit qui valide une saisie fausse et compare la sortie à un texte attendu, en `Locale.ROOT`.

☐ Relancez le test avec `-Duser.language=fr` : la sortie doit être identique (c'est tout l'intérêt de `Locale.ROOT`).

☐ Ajoutez un identifiant de corrélation (uuid) et vérifiez qu'il apparaît dans la ligne de journal mais pas dans le message utilisateur.

12 Erreurs et pièges : les cas réels

> Douze cas, tous rencontrés en production, tous reproduits. Pour chacun : le code fautif, la sortie réelle, la cause, et le correctif.

#	Piège	Symptôme observable
1	Débordement d'`int`	Un stock ou un total négatif, sans aucune exception
2	Argent en `double`	`5.3999999999999995` à l'affichage
3	`new BigDecimal(0.1)`	55 décimales, montants faux au centime
4	`parseInt` sans `strip()`	« valeur correcte » refusée, message incompréhensible
5	« 5 jours » comptés en écart	Une location de 6 jours acceptée
6	`ResolverStyle.SMART`	Le 30 février devient le 28 février
7	`Arrays.asList` exposé	La liste « validée » change après coup
8	File ou liste non bornée	`OutOfMemoryError` sous charge
9	JSON imbriqué sans limite	`StackOverflowError` sur 10 000 crochets
10	`catch` vide	Total faux, zéro erreur, zéro alerte
11	`return` dans `finally`	L'exception disparaît, la méthode renvoie 0
12	Locale par défaut dans un export	`12,35` au lieu de `12.35` (ou l'inverse)

Piège 1 — Le débordement silencieux

REPRODUCTION

```
int stock = 2_000_000_000;
int ajout = 1_500_000_000;
System.out.println(stock + ajout);
```

SORTIE RÉELLE

```
-794967296
```

Cause : la JLS §4.2.2 le dit explicitement — les opérateurs entiers ne signalent ni débordement ni sous-débordement. **Correctif** : `Math.addExact`, ou un type plus large (`long`), ou `BigInteger` pour des valeurs non bornées.

Piège 2 — L'argent en `double`

REPRODUCTION

```
System.out.println(4.50 * 1.20);
```

SORTIE RÉELLE

```
5.3999999999999995
```

Cause : 4,50 et 1,20 ne sont pas représentables exactement en binaire. **Correctif** : `long` en centimes, ou `BigDecimal` avec `setScale(2, RoundingMode.HALF_UP)` à chaque étape monétaire.

Piège 3 — `new BigDecimal(double)`

REPRODUCTION

```
System.out.println(new BigDecimal(0.1));
System.out.println(BigDecimal.valueOf(0.1));
```

SORTIE RÉELLE

```
0.1000000000000000055511151231257827021181583404541015625
0.1
```

Cause : le constructeur `double` prend la valeur binaire exacte. **Correctif** : `new BigDecimal("0.1")` depuis une saisie, `BigDecimal.valueOf(0.1)` depuis un `double` existant. Jamais `new BigDecimal(0.1)`.

Piège 4 — `parseInt` sans normalisation

REPRODUCTION

```
Integer.parseInt(" 42");
```

SORTIE RÉELLE

```
NumberFormatException : For input string: " 42"
```

Cause : `parseInt` est strict (Javadoc `Integer` : tous les caractères doivent être des chiffres, signe admis en tête seulement). **Correctif** : `strip()` avant conversion, et un message d'erreur orienté utilisateur plutôt que la réutilisation du message du JDK.

Piège 5 — « 5 jours » comptés en écart

REPRODUCTION

```
LocalDate debut = LocalDate.of(2026, 11, 10), fin = debut.plusDays(5);
System.out.println(ChronoUnit.DAYS.between(debut, fin) + " < 5 ? " +
(ChronoUnit.DAYS.between(debut, fin) < 5));
```

SORTIE RÉELLE

```
5 < 5 ? false
```

Cause : 10 → 15 en écart fait 5, mais couvre 6 jours calendaires (10, 11, 12, 13, 14, 15). **Correctif** : `between(...) + 1` pour une durée inclusive, et une méthode nommée `nbJoursCalendaires()` pour que la définition soit visible à l'appel.

Piège 6 — Le 30 février accepté

REPRODUCTION

```
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd/MM/uuuu");
System.out.println(LocalDate.parse("30/02/2026", f));
```

SORTIE RÉELLE

```
2026-02-28
```

Cause : `ResolverStyle.SMART` par défaut, qui corrige la date.
Correctif : `.withResolverStyle(ResolverStyle.STRICT)`, qui donne `Invalid date 'FEBRUARY 30'`.

Piège 7 — La collection exposée

REPRODUCTION

```
List<String> source = new ArrayList<>(List.of("a"));
List<String> validee = source;      // même objet
source.add("intrus");
System.out.println(validee);
```

SORTIE RÉELLE

```
[a, intrus]
```

Cause : un `record` (ou une classe) qui stocke la référence ne copie rien. **Correctif** : `List.copyOf(source)` dans le constructeur compact, et `Arrays.copyOf` pour un tableau.

Piège 8 — La file non bornée

REPRODUCTION

```
while (true) file.add(new byte[10 * 1024 * 1024]);
```

SORTIE RÉELLE (TAS DE 256 MO)

```
OutOfMemoryError : Java heap space
```

Cause : aucune limite haute. **Correctif** : file bornée qui lève `FilePleineException` (chapitre 6.4), plus une taille de corps maximale côté serveur.

Piège 9 — Le JSON imbriqué

REPRODUCTION

```
String bomb = "[".repeat(10_000) + "]".repeat(10_000);
parseurRecursif(bomb);
```

SORTIE RÉELLE

```
refus : profondeur maximale dépassée : 100
```

Cause : un analyseur récursif sans plafond de profondeur consomme la pile → `StackOverflowError`. **Correctif** : plafonner la profondeur avant l'analyse (comptage des `[`/`{`), ou configurer la profondeur maximale du désérialiseur.

Piège 10 — Le `catch` vide

REPRODUCTION

```
int total = 0;
for (String s : List.of("10", "x", "30")) {
    try { total += Integer.parseInt(s); } catch (NumberFormatException e) { }
}
System.out.println(total);
```

SORTIE RÉELLE

```
40
```

Cause : l'erreur est ignorée, le calcul continue. **Correctif** : lever une exception enveloppée avec la valeur fautive, ou compter les rejets et produire un rapport explicite.

Piège 11 — Le `return` dans `finally`

REPRODUCTION

```
static int f() {
    try { throw new IllegalStateException("échec"); }
    finally { return 0; }
}
```

SORTIE RÉELLE

```
resultat = 0
```

Cause : un `return` dans `finally` remplace l'exception en cours. **Correctif** : jamais de `return` ni de `throw` dans un `finally` ; utiliser `try-with-resources` pour les ressources.

Piège 12 — La locale par défaut dans un export

REPRODUCTION

```
System.out.println(String.format("%.2f", 12.3456));                // locale par défaut
System.out.println(String.format(Locale.ROOT, "%.2f", 12.3456));    // explicite
```

SORTIE RÉELLE — LOCALE FR_FR

```
12,35
12.35
```

Cause : `String.format` sans locale utilise `Locale.getDefault()`. **Correctif** : locale explicite partout — `Locale.ROOT` pour les fichiers et API, `Locale.FRANCE` pour l'affichage.

> **Le piège au-dessus de tous les autres**
>
> Aucun de ces douze cas ne produit un message d'erreur clair. C'est leur point commun, et c'est ce qui les rend dangereux : ils produisent une valeur plausible. Une valeur fausse mais plausible est bien plus coûteuse qu'une exception, parce qu'elle voyage silencieusement jusqu'à la facture, au rapport ou au score. Le seul antidote est structurel : un type qui refuse de se construire faux (chapitre 1), et des tests qui attaquent les bornes (chapitre 11).

13 Exercices corrigés et fiche de synthèse

> Neuf exercices, progressifs, tous corrigés et vérifiés sur la JVM. Faites-les dans l'ordre : chacun ajoute une limite à celle du précédent. Le barème indique le temps attendu, pas la difficulté.

#	Énoncé	Ce que vous travaillez
1	Conversion d'un entier borné, sans exception	`OptionalInt`, `strip`, bornes
2	Convertisseur qui accumule les erreurs	Validation multi-champs
3	Règle « 5 jours » avec horloge injectée	Dates inclusives, testabilité
4	Quota glissant par utilisateur	Fenêtre, purge, état partagé
5	Analyseur CSV tolérant aux guillemets	Délimiteurs, échappement
6	Plafond de profondeur sur un document imbriqué	Récursion bornée, refus précoce
7	Tour de jeu : stock, quantité, actions	Ordre des contrôles, état intact
8	Message d'erreur à deux niveaux (utilisateur / journal)	Codes stables, séparation
9	Lecture bornée d'un flux, sans exception qui fuit	Plafonds, EOF, décodage strict

Exercice 1 — Convertir un entier borné (5 points, 10 min)

Énoncé. Écrivez `OptionalInt lireEntier(String brut, int min, int max)` qui accepte les espaces autour, refuse `null`, refuse un texte non numérique, et refuse une valeur hors bornes — sans jamais lever d'exception. Le menu d'un jeu l'utilisera.

CORRIGÉ

```
static OptionalInt lireEntier(String brut, int min, int max) {
    if (brut == null) return OptionalInt.empty();
    try {
        int v = Integer.parseInt(brut.strip());
        return (v >= min && v <= max) ? OptionalInt.of(v) : OptionalInt.empty();
    } catch (NumberFormatException e) {
        return OptionalInt.empty();
    }
}
```

SORTIE RÉELLE

```
  "3"   -> OptionalInt[3]
  " 5 " -> OptionalInt[5]
  "6"   -> OptionalInt.empty
  "0"   -> OptionalInt.empty
  "x"   -> OptionalInt.empty
  null  -> OptionalInt.empty
```

Barème : `strip` 1 pt · `null` géré 1 pt · `NumberFormatException` attrapée 1,5 pt · bornes 1,5 pt. **Piège** : ne pas confondre « format » et « bornes » — ici on les fond volontairement dans un même résultat vide, ce qui est acceptable pour un menu mais pas pour un formulaire (exercice 2).

Exercice 2 — Accumuler les erreurs (10 points, 20 min)

Énoncé. Écrivez `record Reservation(LocalDate debut, int jours, int articles)` avec 1 ≤ jours ≤ 5 et 1 ≤ articles ≤ 5, et une fabrique `depuisSaisie(String debut, String jours, String articles)` qui renvoie toutes les violations en une fois.

CORRIGÉ — LES POINTS CLÉS

```
static Reservation depuisSaisie(String debut, String jours, String articles) {
    List<String> violations = new ArrayList<>();
    LocalDate d = null;
    try {
        d = LocalDate.parse(debut == null ? "" : debut.strip());
    } catch (DateTimeParseException e) {
        violations.add("debut [FORMAT_INVALIDE] format attendu AAAA-MM-JJ");
    }
    int j   = entier(jours, "jours", 1, 5, violations);
    int art = entier(articles, "articles", 1, 5, violations);
    if (!violations.isEmpty()) throw new ValidationException(violations);
    return new Reservation(d, j, art);        // le constructeur revalide : ceinture et bretelles
}

static int entier(String s, String champ, int min, int max, List<String> v) {
    int val;
    try { val = Integer.parseInt(s == null ? "" : s.strip()); }
    catch (NumberFormatException e) {
        v.add(champ + " [FORMAT_INVALIDE] entier attendu : \"" + s + "\"");
        return min;
    }
    if (val < min || val > max) v.add(champ + " [VALEUR_HORS_BORNES] entre " + min + " et " + max
+ ", reçu " + val);
    return val;
}
```

SORTIE RÉELLE

```
  Reservation[debut=2026-11-12, jours=5, articles=5]
  refus : [jours [VALEUR_HORS_BORNES] entre 1 et 5, reçu 9, articles [VALEUR_HORS_BORNES] entre 1 et
5, reçu 0]
  refus : [debut [FORMAT_INVALIDE] format attendu AAAA-MM-JJ, articles [FORMAT_INVALIDE] entier
attendu : "x"]
```

Barème : constructeur validant 2 pts · deux violations distinctes sur une même saisie 4 pts · distinction format/bornes 2 pts · aucune `IllegalArgumentException` qui sorte de la fabrique 2 pts.
Piège : si `entier` retourne `0` en cas d'erreur, le constructeur lèvera « jours : 1 à 5, reçu 0 » — d'où l'intérêt de ne construire qu'après avoir vérifié la liste, et de retourner `min` comme valeur neutre.

Exercice 3 — La règle « 5 jours » (10 points, 15 min)

Énoncé. Écrivez `boolean accepte(LocalDate debut, LocalDate fin, Clock horloge)` qui refuse une date passée, une fin antérieure au début, et une durée de plus de 5 jours calendaires. Le test doit passer en 2026 comme en 2030.

CORRIGÉ

```
static boolean accepte(LocalDate debut, LocalDate fin, Clock horloge) {
    LocalDate aujourdhui = LocalDate.now(horloge);
    if (debut.isBefore(aujourdhui)) return false;
    if (fin.isBefore(debut)) return false;
    return ChronoUnit.DAYS.between(debut, fin) + 1 <= 5;
}
```

SORTIE RÉELLE (HORLOGE FIGÉE AU 2026-11-10)

```
  10→14 (5 j)     : true
  10→15 (6 j)     : false
  09 (passé)      : false
  15→10 (inversé) : false
```

Barème : horloge injectée 3 pts · `+1` inclusif 3 pts · fin avant début 2 pts · date passée 2 pts. **Piège** : sans le `+1`, la ligne 10→15 renvoie `true` et vous acceptez 6 jours.

Exercice 4 — Quota glissant (15 points, 25 min)

Énoncé. Deux demandes par 24 h et par utilisateur, fenêtre glissante. À la 3e, lever une exception d'état. Une demande de la veille ne doit plus compter.

CORRIGÉ — L'ESSENTIEL

```
void consommer(String u, LocalDateTime maintenant) {
    Deque<LocalDateTime> dq = historique.computeIfAbsent(u, k -> new ArrayDeque<>());
    while (!dq.isEmpty() && Duration.between(dq.peekFirst(), maintenant).compareTo(fenetre) >= 0)
{
        dq.removeFirst();                       // purge de la fenêtre
    }
    if (dq.size() >= max) throw new IllegalStateException("quota de " + max + " par " +
fenetre.toHours() + " h atteint");
    dq.addLast(maintenant);
}
```

SORTIE RÉELLE

```
  essai 1 accepté, reste 1
  essai 2 accepté, reste 0
  essai 3 refusé : quota de 2 par 24 h atteint
  +24h : reste 1
```

Barème : purge correcte 6 pts · comparaison `>=` sur la fenêtre (et non `>`) 4 pts · exception d'état et non d'argument 3 pts · compteur par utilisateur 2 pts. **Prolongement** : que se passe-t-il avec deux instances de l'application ? (Voir 6.2 : le quota doit être partagé.)

Exercice 5 — Analyseur CSV (15 points, 25 min)

Énoncé. Écrivez `List<String> decouperCsv(String ligne)` qui respecte les guillemets : un `;` entre guillemets ne sépare pas, et `""` à l'intérieur d'un champ vaut un guillemet.

CORRIGÉ

```
static List<String> decouperCsv(String ligne) {
    List<String> champs = new ArrayList<>();
    StringBuilder courant = new StringBuilder();
    boolean dansGuillemets = false;
    for (int i = 0; i < ligne.length(); i++) {
        char c = ligne.charAt(i);
        if (dansGuillemets) {
            if (c == '"') {
                if (i + 1 < ligne.length() && ligne.charAt(i + 1) == '"') { courant.append('"');
i++; }
                else dansGuillemets = false;
            } else courant.append(c);
        } else if (c == '"') {
            dansGuillemets = true;
        } else if (c == ';') {
            champs.add(courant.toString()); courant.setLength(0);
```

```
        } else courant.append(c);
    }
    champs.add(courant.toString());
    return champs;
}
```

SORTIE RÉELLE

```
  "Café;4"              -> [Café, 4]
  "\"Lait \"\"bio\"\"\";10" -> [Lait "bio", 10]
  "\"Thé, vert\";9"      -> [Thé, vert, 9]
```

Barème : état « dans les guillemets » 6 pts · guillemet doublé 4 pts · délimiteur protégé 3 pts · dernier champ ajouté 2 pts. **Prolongement** : ajoutez un plafond de longueur de champ (par exemple 4 096) et de nombre de champs (par exemple 50), en levant une exception au-delà.

Exercice 6 — Profondeur maximale (10 points, 15 min)

Énoncé. Un document est représenté par des crochets imbriqués. Refusez au-delà de 64 niveaux, avant de provoquer un `StackOverflowError`.

CORRIGÉ — LA VERSION ITÉRATIVE, QUI NE PEUT PAS DÉBORDER

```
static int profondeur(String doc) {
    int max = 0, courant = 0;
    for (char c : doc.toCharArray()) {
        if (c == '[') {
            courant++;
            if (courant > 64) throw new IllegalArgumentException("profondeur maximale dépassée :
64");
            max = Math.max(max, courant);
        } else if (c == ']') {
            courant--;
            if (courant < 0) throw new IllegalArgumentException("crochet fermant sans ouvrant");
        }
    }
    if (courant != 0) throw new IllegalArgumentException("document non équilibré");
    return max;
}
```

SORTIE RÉELLE

```
  "[[[]]]"        -> profondeur 3
  "]"             -> refus : crochet fermant sans ouvrant
  "[["            -> refus : document non équilibré
  10 000 crochets -> refus : profondeur maximale dépassée : 64
```

Barème : compteur borné 4 pts · déséquilibre détecté 3 pts · itératif (pas de récursion) 3 pts. **Leçon** : la version récursive est plus lisible mais consomme la pile de la JVM ; la version itérative a une mémoire constante et vérifie la limite au bon moment.

Exercice 7 — Le tour de jeu (15 points, 20 min)

Énoncé. Un tour autorise au plus 3 actions et un stock initial de 5 potions. Toute tentative invalide doit laisser l'état intact : ni potion consommée, ni action décomptée.

CORRIGÉ — L'ORDRE DES CONTRÔLES EST LE SUJET

```
void consommer(int n) {
    if (n <= 0) throw new IllegalArgumentException("quantité invalide : " + n);
    if (n > potions) throw new IllegalStateException("stock insuffisant : " + n + " demandés, " +
potions + " disponibles");
```

```
    if (actions >= maxActions) throw new IllegalStateException("plus d'action ce tour : " +
maxActions + " maximum");
    potions -= n;
    actions++;
}
```

SORTIE RÉELLE

```
  potions=3 actions=1
  refus : stock insuffisant : 4 demandés, 3 disponibles
  refus : quantité invalide : -1
  potions=3 actions=1 (inchangé)
```

Barème : argument avant état 4 pts · état avant effet 6 pts · deux messages distincts 3 pts · état intact après refus, prouvé par la sortie 2 pts. **Piège** : `potions -= n` avant les contrôles laisse un stock négatif si l'exception part.

Exercice 8 — Deux messages, un seul code (10 points, 15 min)

Énoncé. Pour chaque violation, produisez un message utilisateur en français et une ligne de journal en anglais, à code stable. Le test doit produire la même ligne de journal quelle que soit la machine.

CORRIGÉ

```
record Violation(String champ, CodeErreur code, String messageUtilisateur) {
    String pourUtilisateur() { return messageUtilisateur + " (champ : " + champ + ")"; }
    String pourLogs() {
        return String.format(Locale.ROOT, "code=%s champ=%s msg=\"%s\"", code, champ,
messageUtilisateur);
    }
}
```

SORTIE RÉELLE

```
  utilisateur : La durée maximale est de 5 jours. Vous avez demandé 9 jours. (champ : jours)
  journal     : code=VALEUR_HORS_BORNES champ=jours msg="La durée maximale est de 5 jours. Vous avez
demandé 9 jours."
```

Barème : code stable présent 4 pts · locale explicite 3 pts · message utilisateur sans jargon technique 3 pts. **Leçon** : un journal qui dépend de la locale de la machine rend les alertes inexploitables — la même erreur produit `12,35` ou `12.35` selon l'hôte.

Exercice 9 — Lire un flux borné et rendre un compte exploitable (10 points, 20 min)

Énoncé. Écrivez `Resultat lire(InputStream in, int maxLignes, int maxOctets)` qui ne lève jamais : il lit au plus `maxLignes` lignes de `maxOctets` octets chacune, et rend soit les lignes, soit un message de refus. Testez cinq cas : nominal, trop de lignes, ligne trop longue, octet invalide, flux vide. Vérifiez la borne exacte : 40 octets acceptés, 41 refusés.

CORRIGÉ

```
record Resultat(List<String> lignes, String erreur) {
    static Resultat ok(List<String> l) { return new Resultat(l, null); }
    static Resultat refus(String e) { return new Resultat(List.of(), e); }
}

static Resultat lire(InputStream in, int maxLignes, int maxOctets) {
    try {
        return Resultat.ok(LecteurBorne.depuis(in, StandardCharsets.UTF_8)
                .lire(maxLignes, maxOctets));
```

```
    } catch (FluxTropLongException e) {
        return Resultat.refus(e.getMessage());
    }
}
```

SORTIE RÉELLE

```
1. nominal       : OK [LOC-1;5, LOC-2;2]
2. trop de lignes: REFUS plus de 3 lignes : maximum 3 (21 octets déjà lus)
3. ligne longue  : REFUS ligne trop longue : 306 octets, maximum 40
4. octet Latin-1 : REFUS octet non décodable en UTF-8
5. flux vide     : OK []

borne exacte : 3 lignes de 40 octets acceptées, 41 refusées
  OK [0123456789012345678901234567890123456789]
  REFUS ligne trop longue : 41 octets, maximum 40
```

Barème : aucune exception qui fuit 4 pts · la borne exacte testée des deux côtés 3 pts · flux vide traité comme un succès 2 pts · message de refus exploitable 1 pt. **Leçon** : la borne se teste toujours à `max` et à `max + 1` — un test au seul cas nominal ne prouve rien. Et un flux vide n'est pas une erreur : le refuser oblige les appelants à traiter un cas qui n'en est pas un.

Fiche de synthèse

Besoin	Solution	Piège associé
Limiter une valeur	Constructeur compact du `record`, constante `public static final`	Un contrôle dans le contrôleur seulement : contourné ailleurs
Limiter une durée en jours	`ChronoUnit.DAYS.between(a, b) + 1`	Oublier le `+1` : 6 jours acceptés
Limiter un nombre	`Math.addExact`, `multiplyExact`, `toIntExact`	Opérateur nu : débordement silencieux
Représenter de l'argent	`long` en centimes ou `BigDecimal` + `RoundingMode`	`double`, ou `new BigDecimal(0.1)`
Parser une date	`ISO_LOCAL_DATE`, ou motif `uuuu` + `ResolverStyle.STRICT`	`SMART` : le 30 février devient le 28
Tester le temps	`Clock` injectée (`Clock.fixed`)	`LocalDate.now()` : intestable
Limiter un volume	Plafond en éléments et en octets, vérifié avant lecture	`long` oublié dans le produit : négatif, donc accepté
Limiter un rythme	Fenêtre glissante ou seau à jetons	État local : 3 instances = 3 × le quota
Signaler une saisie	Violations accumulées (champ, code, message)	Tout en un message global : impossible à corriger
Signaler une panne	Exception enveloppée, cause conservée, journal	`catch` vide, `finally` avec `return`
Lire une entrée	Encodage et délimiteur explicites, plafonds de lignes et de longueur, fin de flux traitée comme un cas normal	Encodage implicite, BOM non nettoyé, `readLine() == null` ignoré
Cadrer l'output	Format fixe, locale explicite, échappement CSV, UTF-8	Largeur prise pour un maximum, locale par défaut

Les onze règles à retenir

- ☐ **Un objet construit est un objet valide.** La validation va dans le type, pas dans l'appelant.
- ☐ **Une limite métier est une constante nommée**, jamais un `5` recopié à trois endroits.
- ☐ **Deux limites par entrée** : la règle métier, et le plafond technique qui protège la machine.
- ☐ **Vérifiez avant d'allouer** : taille, profondeur, nombre d'éléments — jamais après.
- ☐ `Math.*Exact` **et** `BigDecimal` : l'arithmétique silencieuse est l'ennemi.
- ☐ **Injectez l'horloge** : une règle qui dépend du temps doit être testable.
- ☐ **Accumulez les erreurs de saisie**, mais bornez-en le nombre et nommez chaque champ.
- ☐ **N'attrapez jamais pour ignorer.** Enveloppez, conservez la cause, journalisez, relancez.
- ☐ **Deux messages pour une erreur** : celui de l'utilisateur et celui du journal, avec un code stable.
- ☐ **Une lecture a toujours trois paramètres** : la source, l'encodage, le plafond. Ce qui n'y est pas sera appliqué ailleurs, ou pas du tout.
- ☐ **Méfiez-vous de l'output autant que de l'input** : locale, encodage, délimiteurs, largeurs.

La réponse en trois phrases

Limiter une entrée : un invariant dans le constructeur du type qui la porte, doublé d'une conversion qui accumule les erreurs à la frontière. **Gérer les erreurs** : une exception typée avec un code et un message construits, jamais avalée, toujours enveloppée avec sa cause. **Lire l'entrée** : un lecteur choisi selon la source, un encodage explicite, et des plafonds dans la signature. **Cadrer l'output** : un format unique, une locale explicite, un encodage explicite, et deux versions de chaque message — utilisateur et journal.

14 Sources et pour aller plus loin

> Toutes les affirmations de ce document viennent de la spécification du langage, de la Javadoc du JDK 25, ou d'une exécution réelle. Voici de quoi vérifier et approfondir.

Spécification du langage Java (SE 25)

Section	Ce qu'elle établit, et où c'est utilisé ici
§4.2.2 « Integer Operations »	« The integer operators do not indicate overflow or underflow in any way. » C'est la base du chapitre 2.1.
§5.1.3 « Narrowing Primitive Conversion »	Une conversion rétrécissante ne lève jamais d'exception, même en cas de perte d'information. D'où la nécessité de `Math.toIntExact`.
§11 « Exceptions »	Définition des exceptions vérifiées et non vérifiées (§11.1.1), et la règle de compilation des `catch` (§11.2.3).
§14.10 « The assert Statement »	Les assertions sont activées ou désactivées ; par défaut, désactivées. D'où la section 7.6.
§14.20.3 « try-with-resources »	Le mécanisme d'exception supprimée (§14.20.3.2) : `addSuppressed`. Section 7.5.
§15.17.2 et §15.17.3	Division et reste entiers : `ArithmeticException` si le diviseur est nul, sinon troncature vers zéro. Section 2.4.
§10.7 « `static` Fields »	`System.in`, `System.out`, `System.err` sont des champs statiques modifiables : ce sont des flux, pas des constantes. Section 9.9.
§15.18.2	Additions numériques et les cas spéciaux des flottants (Infinity, NaN). Section 2.5.

Javadoc JDK 25 — les classes utilisées

Classe / interface	À lire en priorité
`java.lang.Math`	Les méthodes `*Exact` et le paragraphe qui les justifie (« the methods whose names end with Exact throw an ArithmeticException when the results overflow ») ; `floorDiv` / `floorMod`.
`java.lang.Integer`	`parseInt(String)` : la liste exacte des cas de `NumberFormatException`. Section 2.7.
`java.math.BigDecimal`	`BigDecimal(double)` (« the exact decimal representation of the double's binary floating-point value »), `divide(…, scale, RoundingMode)`, `setScale`.
`java.math.RoundingMode`	Les huit modes ; en pratique `HALF_UP` pour l'affichage et la facturation, `HALF_EVEN` pour les calculs financiers longs.
`java.time.LocalDate`, `ChronoUnit`	`between` (exclusif), `plusDays` (calendaire) ; les bornes `MIN` / `MAX`.
`java.time.Clock`	`fixed`, `systemUTC`, `offset` : la base de la testabilité temporelle.
`java.time.format.DateTimeFormatter` et `ResolverStyle`	`ofPattern`, `withResolverStyle`, et les trois résolveurs `STRICT` / `SMART` / `LENIENT`.
`java.util.Objects`	`requireNonNull` (avec message), `requireNonNullElse`, `toString(obj, défaut)`, `checkIndex`.
`java.util.Scanner`	Le délimiteur par défaut (« matches whitespace »), `useDelimiter`, `useLocale`.
`java.util.Formatter`	Les spécificateurs, les largeurs, et les exceptions de format (`IllegalFormatConversionException`, `MissingFormatArgumentException`).
`java.nio.charset.Charset`, `StandardCharsets`	`defaultCharset`, et la différence entre `file.encoding`, `stdout.encoding` et `native.encoding`.
`java.nio.file.Files`	`size` avant lecture, `readString(path, charset)`, `newInputStream`.
`java.io.BufferedReader`, `InputStreamReader`	`readLine()` qui rend `null` en fin de flux ; le constructeur avec `CharsetDecoder` pour choisir `REPORT` ou `REPLACE`. Chapitre 9.
`java.nio.charset.CharsetDecoder`, `CodingErrorAction`	`onMalformedInput`, `onUnmappableCharacter` : les deux politiques de décodage, et l'exception associée. Section 9.5.
`java.io.Console`	`System.console()` qui peut valoir `null` ; `readPassword()` pour une saisie sans écho. Section 9.9.
`java.util.regex.Pattern`	`hasNext(Pattern)` / `next(Pattern)` : valider la forme pendant la lecture. Section 9.3.
`java.nio.file.Files.lines`	Flux paresseux qui garde le fichier ouvert : `try-with-resources` obligatoire. Section 9.6.
`java.security.SecureRandom`, `java.util.random.RandomGenerator`	La différence entre reproductible et imprévisible ; `RandomGeneratorFactory` et les graines explicites.
`java.util.Properties`	`load(InputStream)` en ISO-8859-1, `load(Reader)` avec un encodage choisi, `getProperty(clé, défaut)`. Chapitre 10.5.
`org.yaml.snakeyaml` 2.3	`SafeConstructor`, `LoaderOptions.setMaxAliasesForCollections`, le comportement des types implicites. Chapitre 10.2 à 10.8.
`org.tomlj` 1.1.1	`Toml.parse`, `hasErrors()`, `errors()` avec position, accès typé `getLong` / `getLocalDate`. Chapitre 10.4.
`java.text.Normalizer`	Les formes NFC / NFD, et pourquoi deux chaînes identiques à l'œil peuvent différer.

Textes normatifs utiles hors du JDK

Référence	Sujet
RFC 4180	Format CSV : guillemets, guillemets doublés, fins de ligne. Section 9.3.
RFC 9457 (« Problem Details for HTTP APIs »)	Format normalisé d'un corps d'erreur d'API : `type`, `title`, `status`, `detail`, `instance`. Section 9.5.
OWASP « Input Validation »	Validation en liste blanche, canonisation, injection CSV, taille maximale des entrées.
OWASP « Deserialization Cheat Sheet »	Profondeur et taille des documents, types autorisés, refus des objets arbitraires.
YAML 1.1 / 1.2 (yaml.org)	Le tableau des types implicites, dont les booléens `y|Y|yes|no|NO|on|off` et les dates non guillemetées. Chapitre 10.2.
TOML v1.0.0 (toml.io)	Les types explicites : entiers sans zéro initial, flottants avec chiffres des deux côtés, dates et heures, interdiction des clés dupliquées. Chapitre 10.4.
CVE-2022-1471 (SnakeYAML)	Désérialisation arbitraire via `Constructor` : versions `>= 0 < 2.0` affectées, corrigées en 2.0 ; `SafeConstructor` et limite d'alias. Section 10.8.
CWE-674 / « billion laughs »	Déni de service par références récursives : la limite `setMaxAliasesForCollections` de SnakeYAML. Section 10.8.
JEP 400 (Java 18)	UTF-8 par défaut pour les API de la bibliothèque standard : `file.encoding` vaut désormais UTF-8.
RFC 3629 (UTF-8)	Forme valide d'une séquence UTF-8 : c'est elle qu'un décodeur strict refuse de violer. Section 9.5.
JEP 306 (Java 17)	Arithmétique à virgule flottante stricte : plus de différence entre `double` et `strictfp`.

Pour aller plus loin, par sujet

Sujet	Par où commencer
Validation déclarative	Jakarta Bean Validation (`@NotNull`, `@Min`, `@Max`, `@Size`) : utile à la frontière HTTP, mais ne remplace pas les invariants du domaine — les deux se complètent.
Types qui se valident eux-mêmes	Le motif « Value Object » : un `record` par concept (Email, Quantite, PlageDates) plutôt que des `String` et des `int` nus partout.
Tests de limites	Analyse aux limites : pour chaque borne, tester `min-1`, `min`, `max`, `max+1`. C'est le jeu de tests qui trouve les exercices 1 à 7 de ce chapitre.
Tests de propriétés	jqwik : générer des milliers d'entrées aléatoires et vérifier qu'aucune ne produit d'objet invalide — trouve les débordements que les exemples manuels oublient.
Limitation de débit en production	Au niveau du répartiteur ou d'une passerelle (par utilisateur, par IP), avec un état partagé ; le code du chapitre 6 en est la version pédagogique.
Analyse d'arguments en production	Les bibliothèques dédiées (picocli, JCommander) gèrent options, sous-commandes, aide et messages d'erreur à votre place ; le chapitre 9.7 montre ce qu'elles font, pour savoir ce qu'on délègue.
Configuration typée au démarrage	Les frameworks (Spring Boot `@ConfigurationProperties`, SmallRye Config, MicroProfile Config) font la fusion des sources et la conversion ; la validation, elle, reste votre affaire — c'est le chapitre 10.7.
Observabilité des rejets	Compter les refus par code (`VALEUR_HORS_BORNES`, `QUOTA_DEPASSE`…) : un pic de refus est souvent le premier signe d'un bug côté client.

Comment vérifier ce document

Toutes les sorties présentées ont été produites par des programmes compilés avec `javac --release 21` et exécutés sur Temurin 25.0.5, sous Debian 13, en novembre 2026. Les fichiers sources sont fournis avec la leçon (répertoire `demo/`) et se compilent en une commande : `javac --release 21 -d out $(find src -name '*.java')`. Les cas marqués « avec `-Xmx256m` » ou « avec `-ea` » nécessitent explicitement ces options — c'est justement leur intérêt.